

Chapter 7 – Logical Agents

AI 501 - 2025

Outline

Knowledge-based agents

Wumpus world

Logic: models and entailment

Propositional (Boolean) logic

Equivalence, validity, satisfiability

Inference rules and theorem proving

- resolution
- forward chaining
- backward chaining

DPLL Algorithm

Effective Propositional Model Checking

Knowledge based agents...

... reason over an internal **representation** of knowledge to decide what actions to take.

- can accept **new tasks** in the form of explicitly described goals
- can **achieve competence** quickly by being told or learning new knowledge about the environment
- can **adapt to changes** in the environment by updating the relevant knowledge.

Knowledge base - KB

Set of sentences in a
formal language

Comes with a way to add
new sentences - **TELL**

and a way to query
what is known - **ASK**

KB = **axioms** +
sentences that “follow”
from other sentences

Knowledge level

Describe the agent by
what it knows

KB

domain-specific content

Implementation level

Describing the agent
by what it does

Inference engine

domain-independent algorithms

A simple Knowledge-based agent

```
function KB-Agent(percept) returns an action
  static: KB, a knowledge base
           t, a counter, initially 0, indicating time
  TELL(KB, Make-Percept-Sentence(percept, t))
  action ← ASK(KB, Make-Action-Query(t))
  TELL(KB, Make-Action-Sentence(action, t))
  t ← t + 1
  return action
```

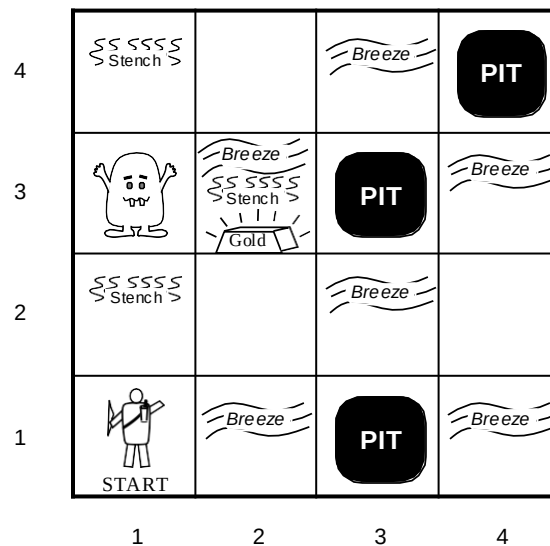
A generic knowledge-based agent. Given a percept, the agent adds the percept to its knowledge base, asks the knowledge base for the best action, and tells the knowledge base that it has in fact taken that action.

Wumpus world

*The wumpus world is a cave consisting of rooms connected by passageways. Lurking somewhere in the cave is the **terrible wumpus**, a beast that eats anyone who enters its room.*

*Some rooms contain bottomless **pits** that will trap anyone who wanders into these rooms (except for the wumpus, which is too big to fall in).*

*The only redeeming feature of this bleak environment is the possibility of finding a heap of **gold**.*



Wumpus world

Environment

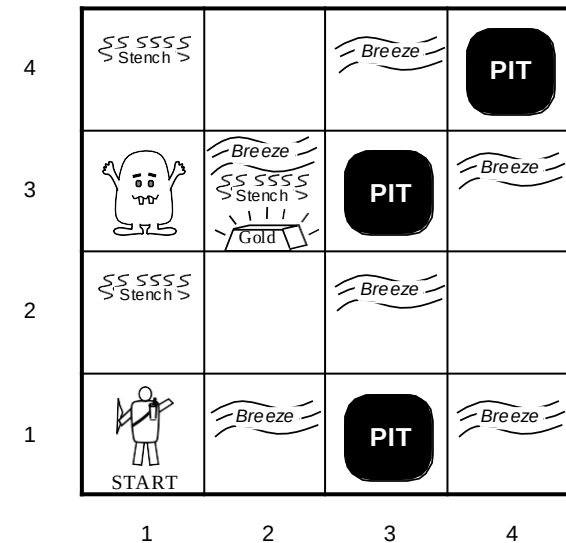
Squares adjacent to wumpus are smelly

Squares adjacent to pit are breezy

Actuators

Left turn, Right turn, Forward

Sensors Breeze, Smell



Exploring a Wumpus world

OK			
OK A	OK		

A= Agent

B= Breeze

G= Glitter

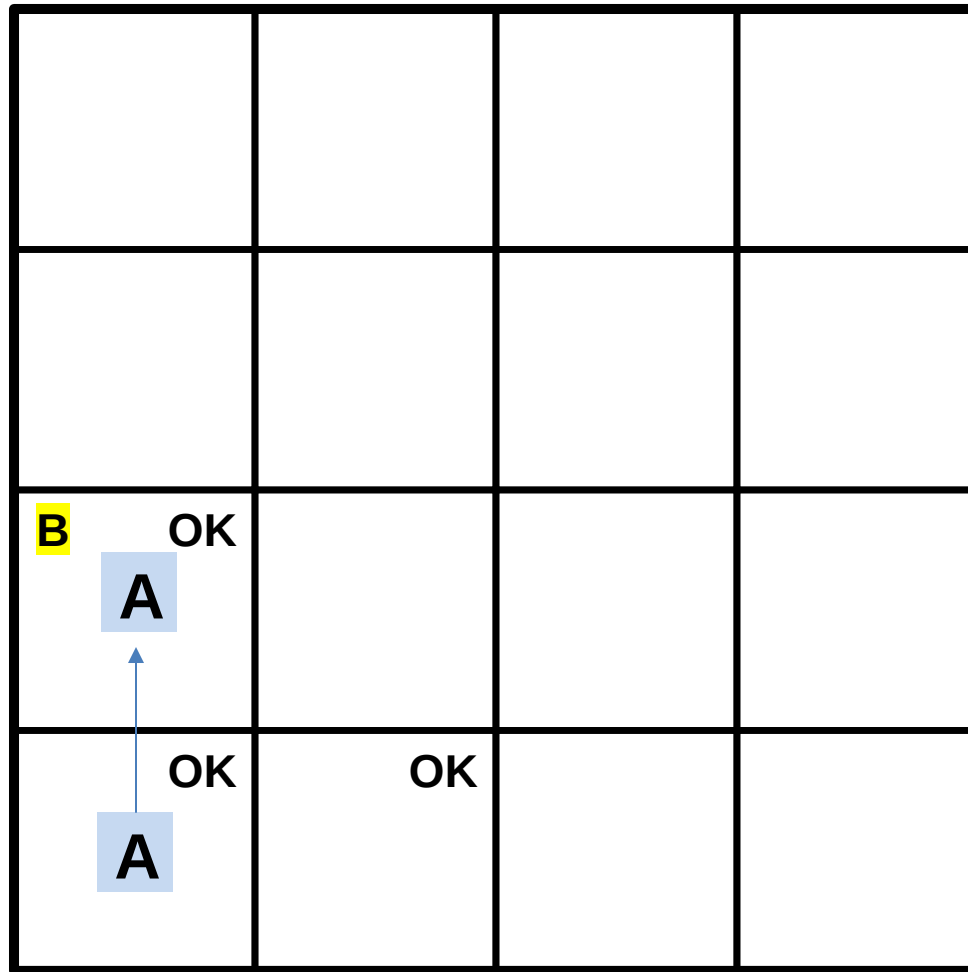
P= Pit

S= Stench

W=Wumpus

OK = Safe

Exploring a Wumpus world



A = Agent

B = Breeze

G = Glitter

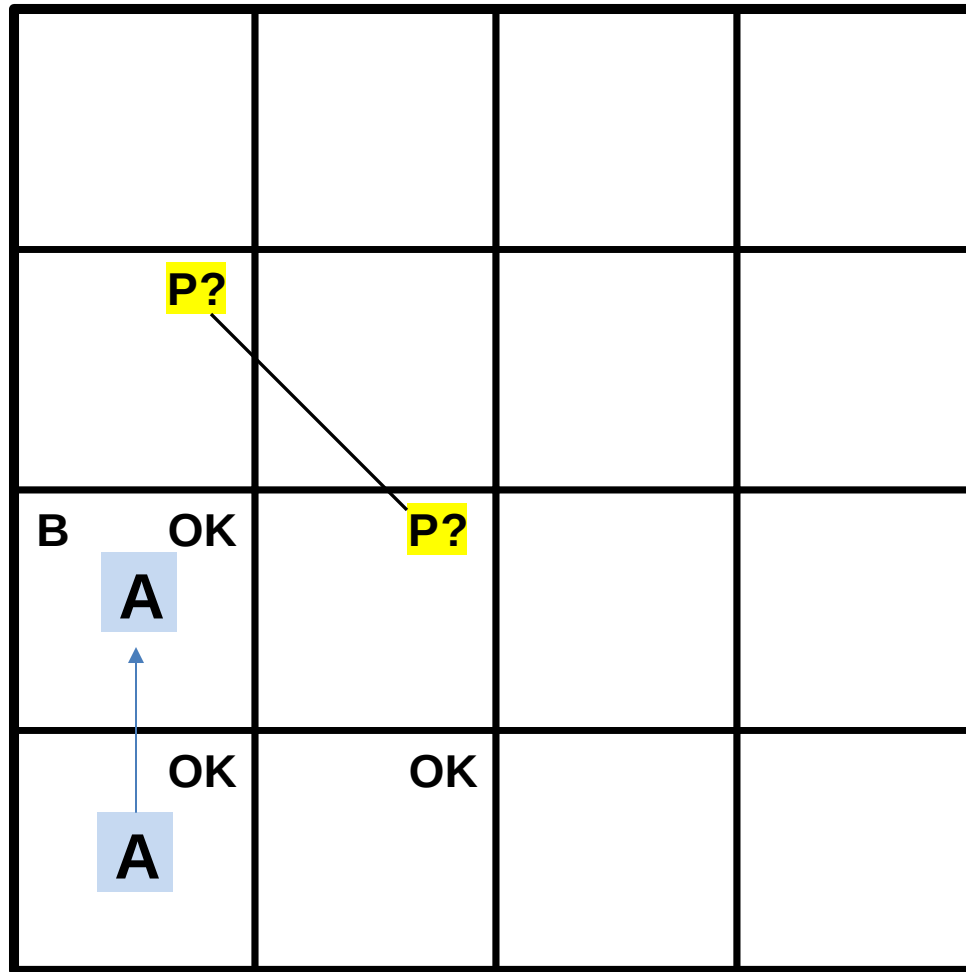
P = Pit

S = Stench

W = Wumpus

OK = Safe

Exploring a Wumpus world



A= Agent

B= Breeze

G= Glitter

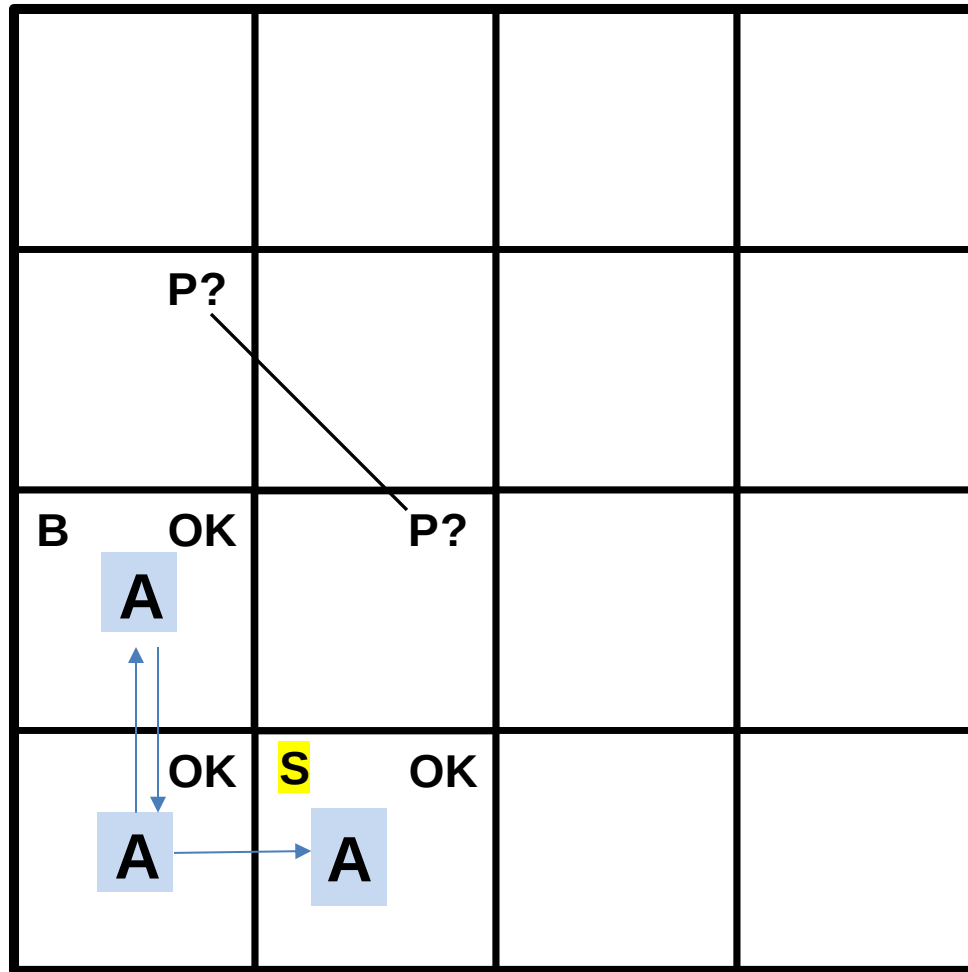
P= Pit

S= Stench

W=Wumpus

OK = Safe

Exploring a Wumpus world



A = Agent

B = Breeze

G = Glitter

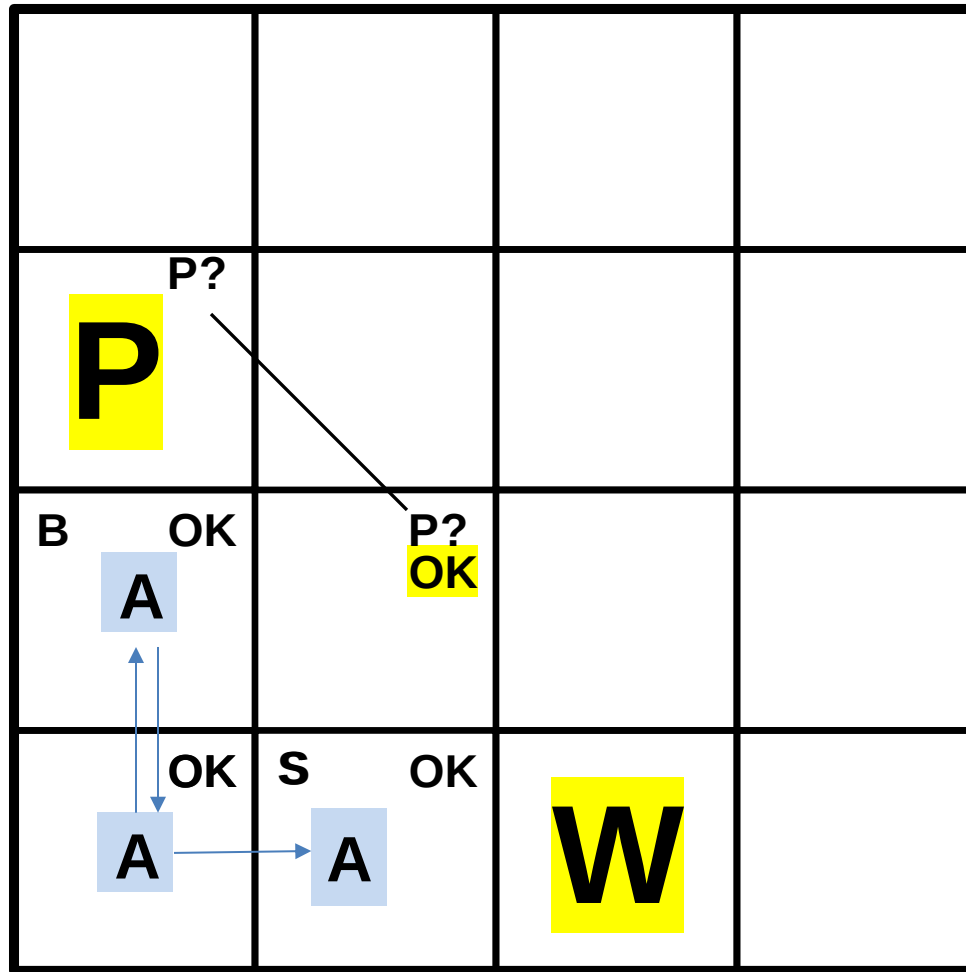
P = Pit

S = Stench

W = Wumpus

OK = Safe

Exploring a Wumpus world



A= Agent

B= Breeze

G= Glitter

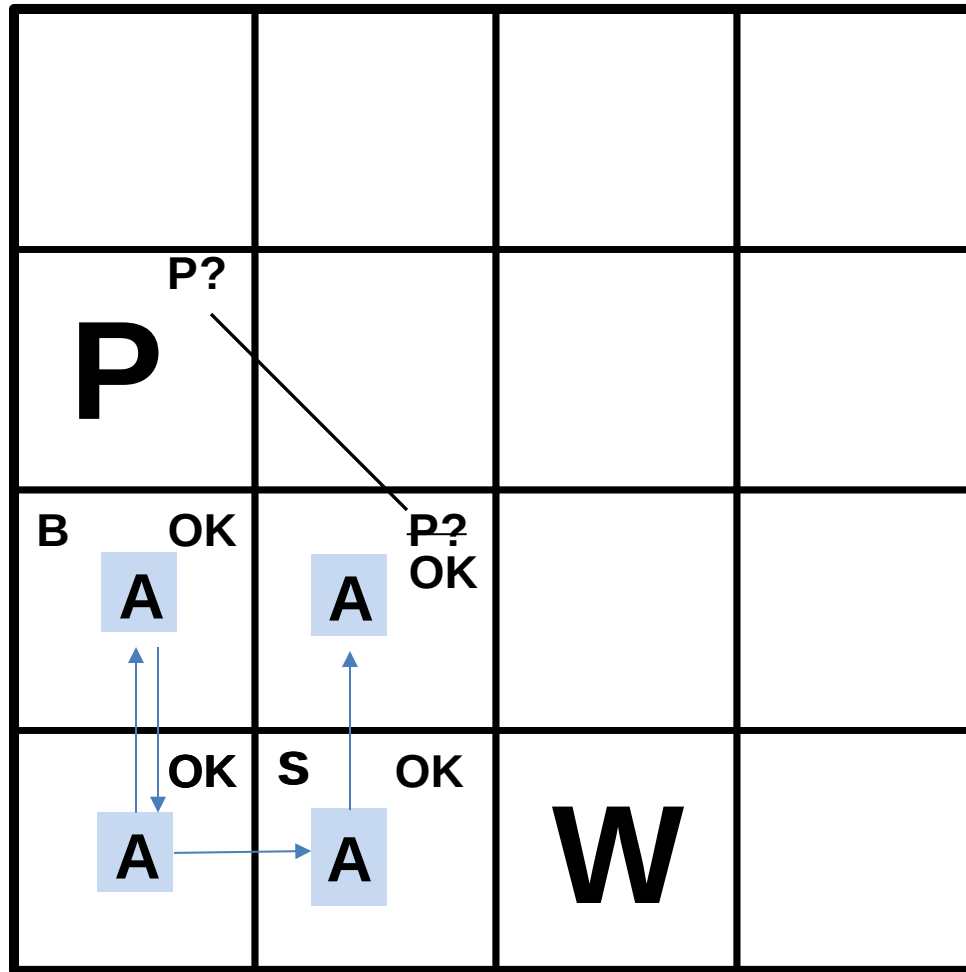
P= Pit

S= Stench

W=Wumpus

OK = Safe

Exploring a Wumpus world



A= Agent

B= Breeze

G= Glitter

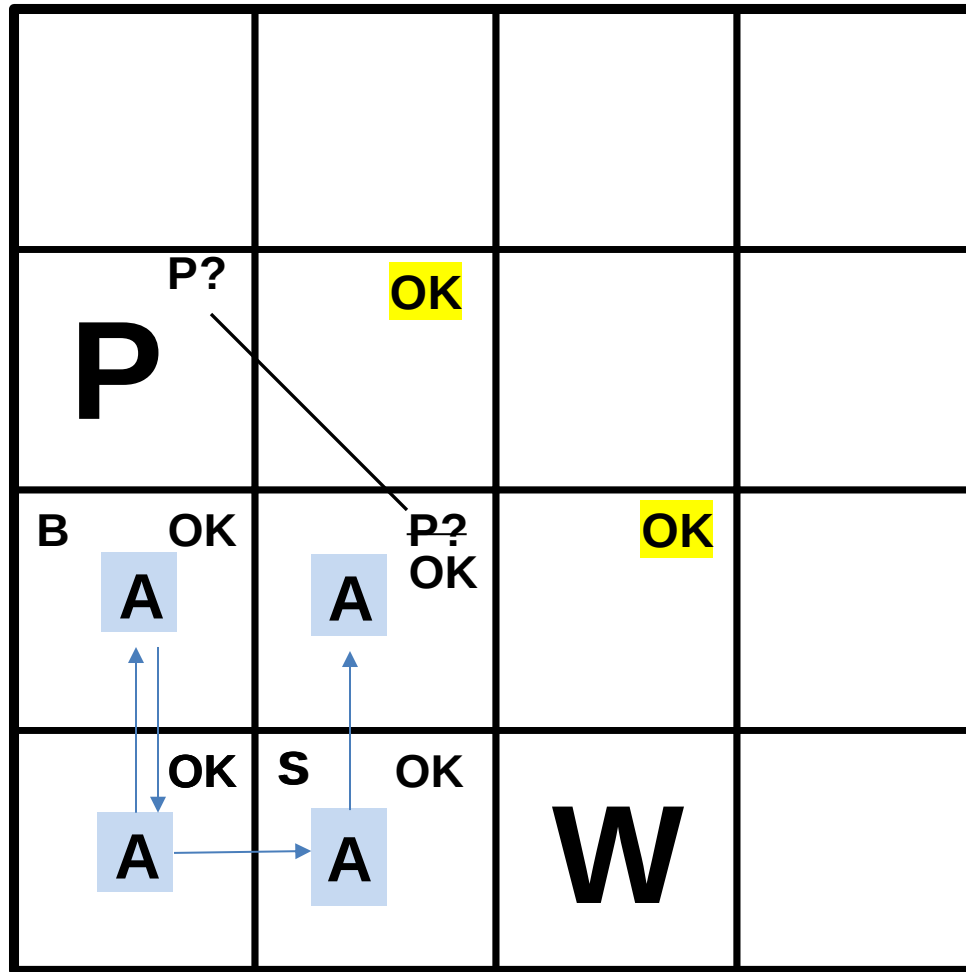
P= Pit

S= Stench

W=Wumpus

OK = Safe

Exploring a Wumpus world



A= Agent

B= Breeze

G= Glitter

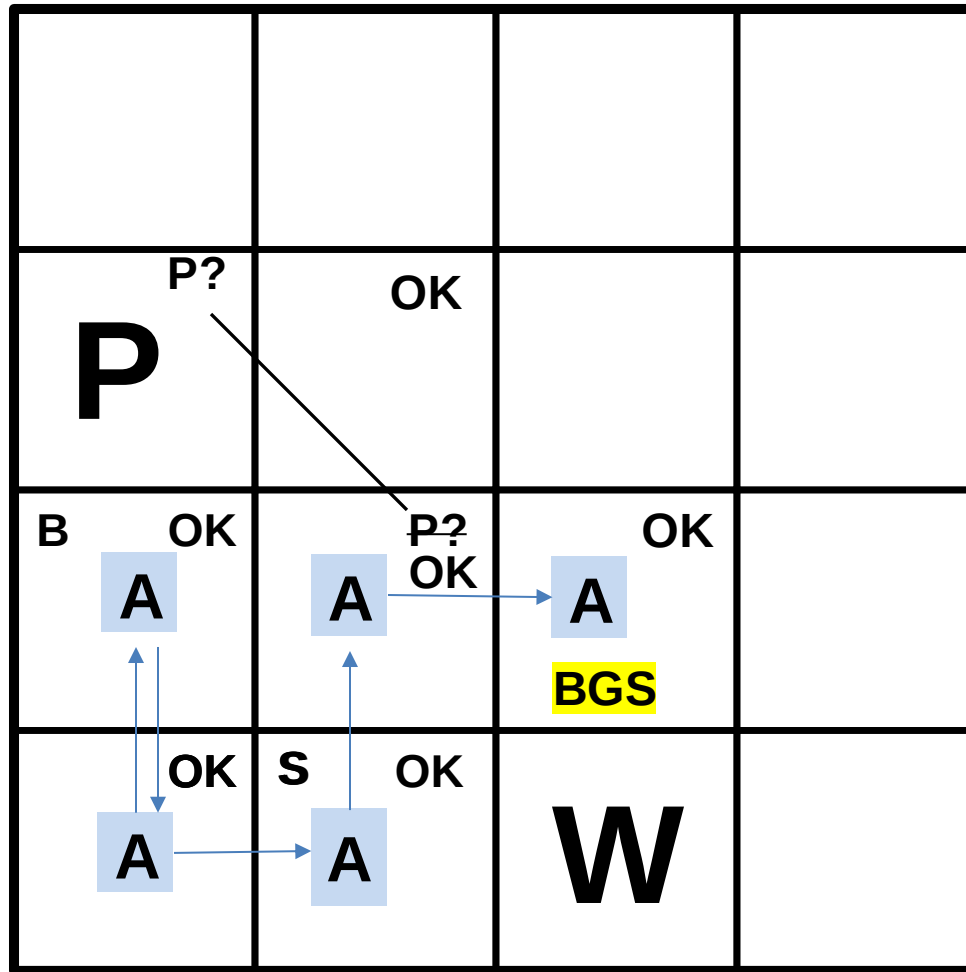
P= Pit

S= Stench

W=Wumpus

OK = Safe

Exploring a Wumpus world



A = Agent

B = Breeze

G = Glitter

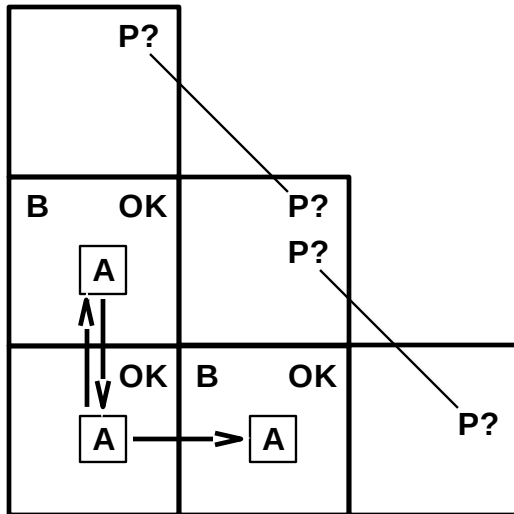
P = Pit

S = Stench

W = Wumpus

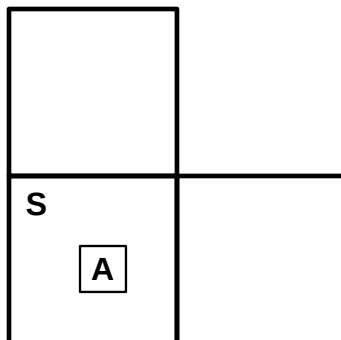
OK = Safe

Other tight spots



Breeze in (1,2) and (2,1)
 $\Rightarrow$ no safe actions

Assuming pits uniformly distributed,
 (2,2) has pit w/ prob 0.86, vs. 0.31



Smell in (1,1)

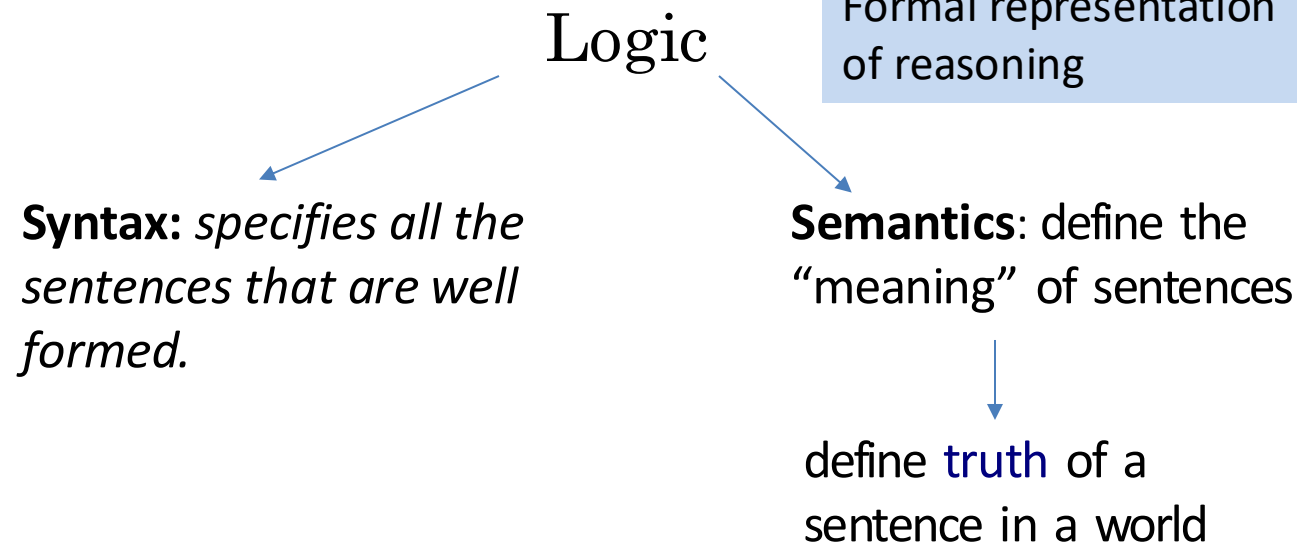
$\Rightarrow$ cannot move

Can use a strategy of **coercion**:

shoot straight ahead

wumpus was there $\Rightarrow$ dead $\Rightarrow$ safe

wumpus wasn't there $\Rightarrow$ safe



E.g., the language of arithmetic

$x + 2 \geq y$ is a sentence; $x^2 + y >$ is not a sentence

$x + 2 \geq y$ is true iff the number $x + 2$ is no less than the number y

$x + 2 \geq y$ is true in a world where $x = 7$, $y = 1$

$x + 2 \geq y$ is false in a world where $x = 0$, $y = 6$

Entailment

$$KB \models a$$

Knowledge base KB entails sentence a
if and only if
 a is true in all worlds where KB is true

E.g., $x + y = 4$ entails $4 = x + y$

Entailment is a relationship between sentences (i.e., **syntax**)
that is based on **semantics**

Note: brains process **syntax** (of some sort)

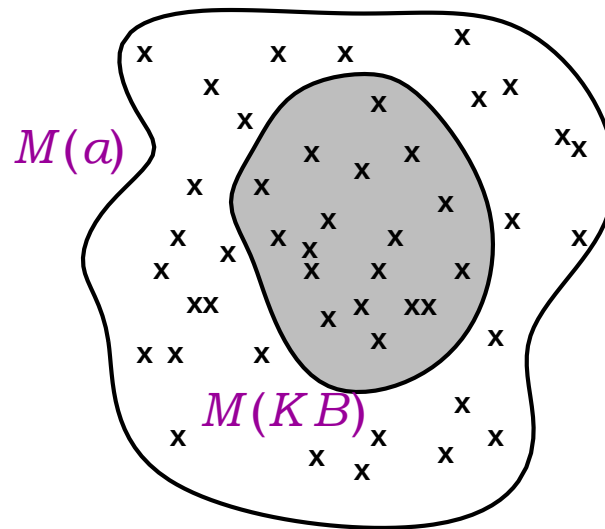
Models

formally structured worlds with respect to which truth can be evaluated

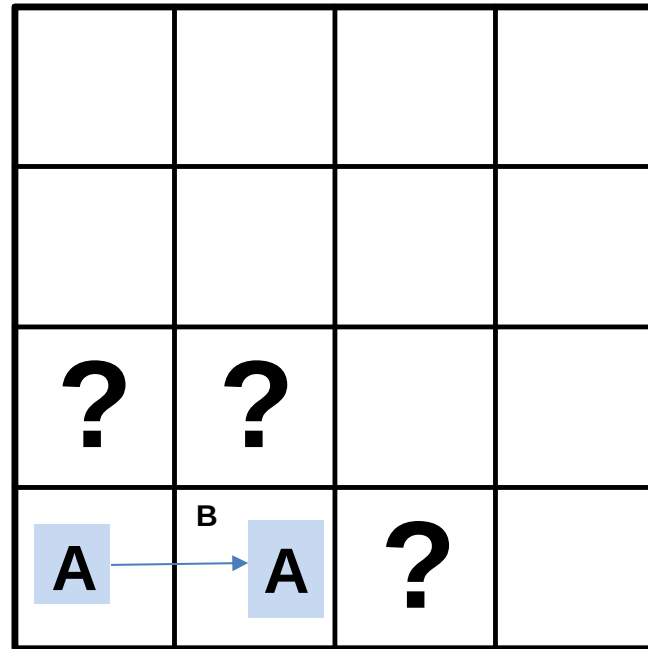
We say m is a model of a sentence a if a is true in m

$M(a)$ is the set of all models of a

Then $KB \models a$ if and only if $M(KB) \subseteq M(a)$

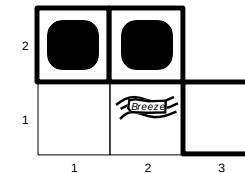
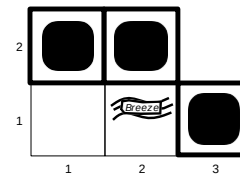
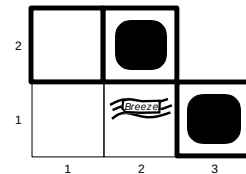
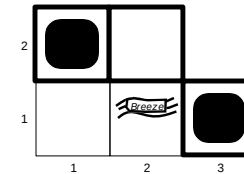
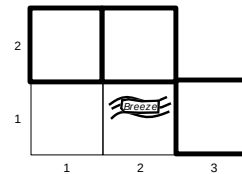
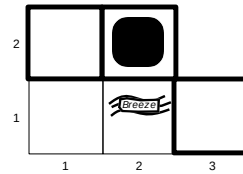
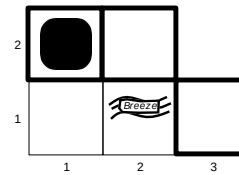
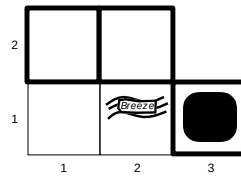


Entailment in the Wumpus world

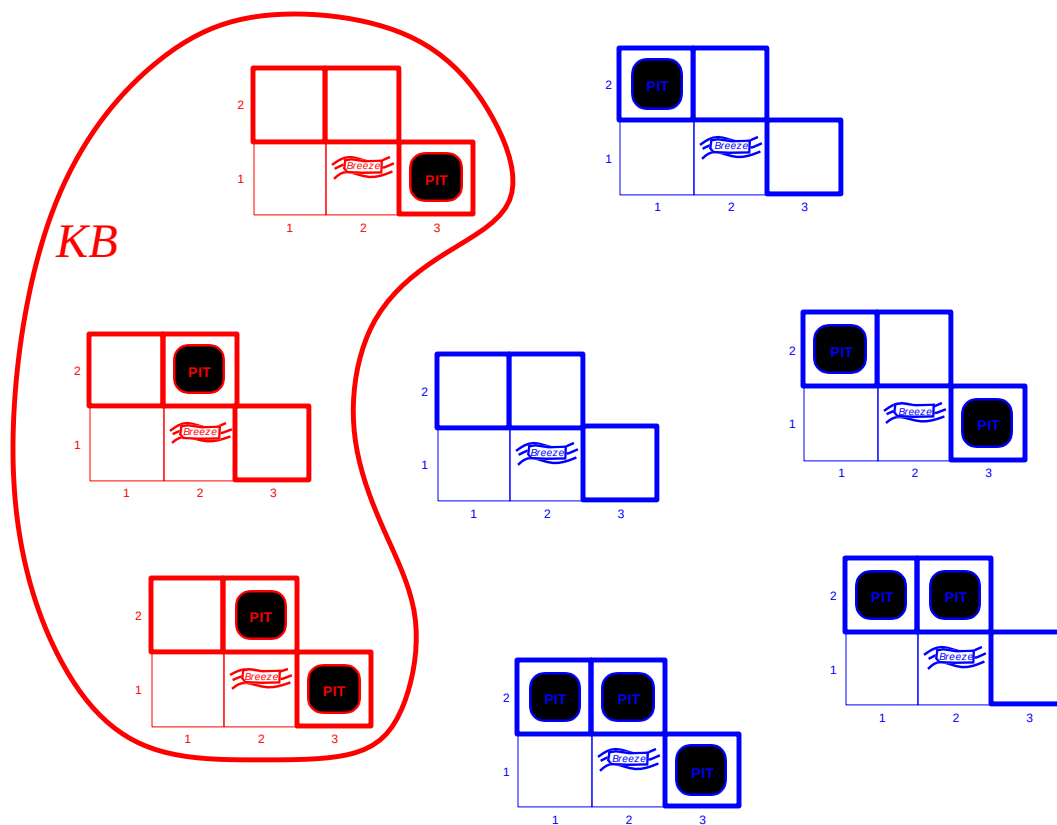


There are 8 possible models

Wumpus models

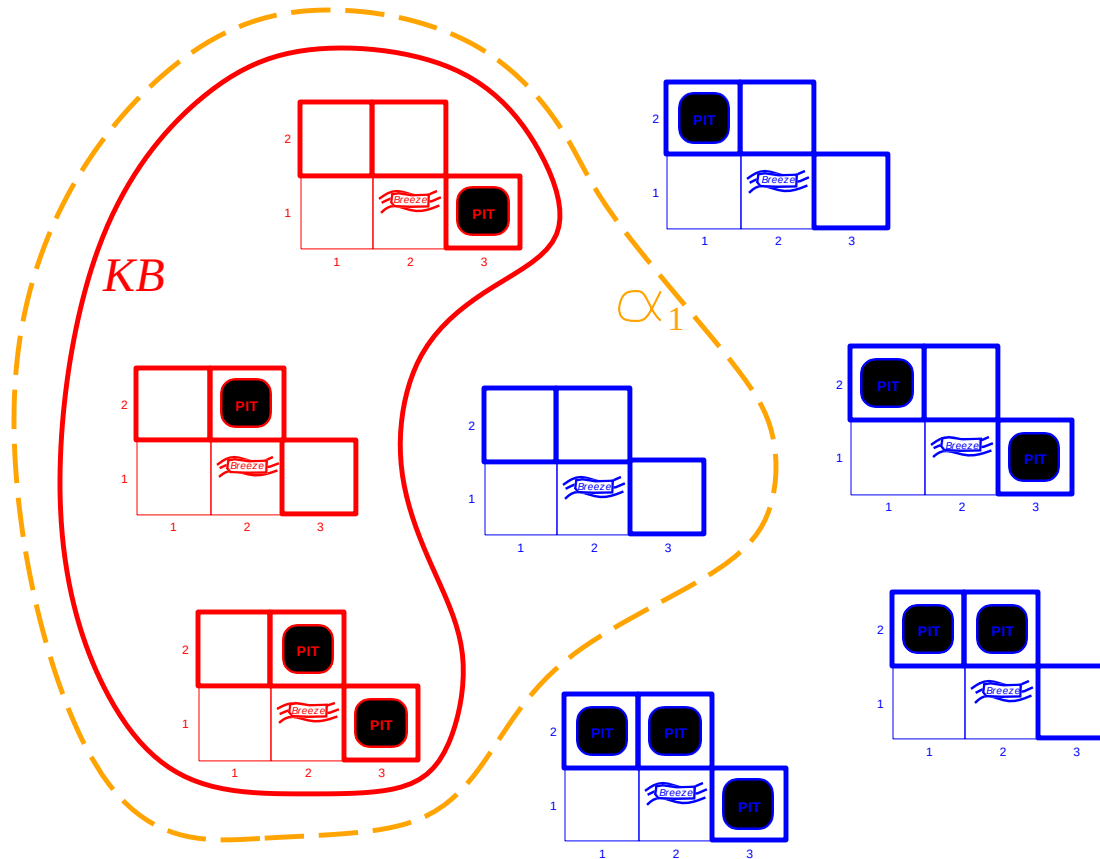


Wumpus models



KB = wumpus-world rules + observations

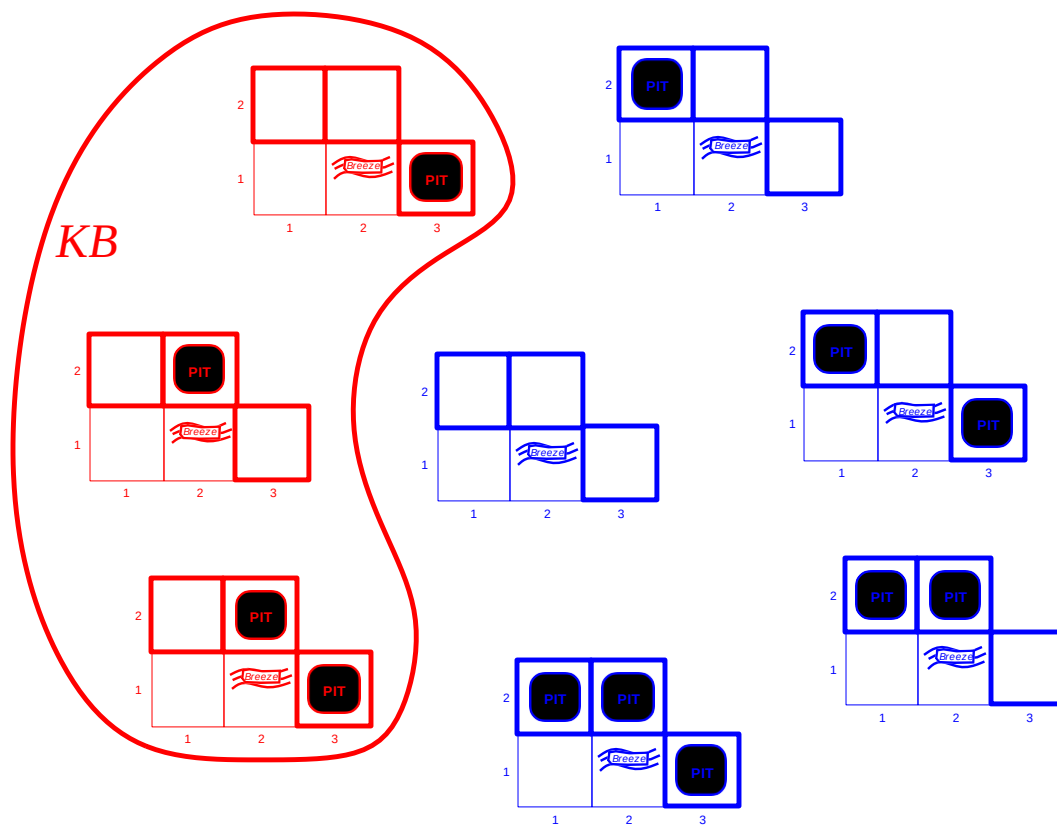
Wumpus models



KB = wumpus-world rules + observations

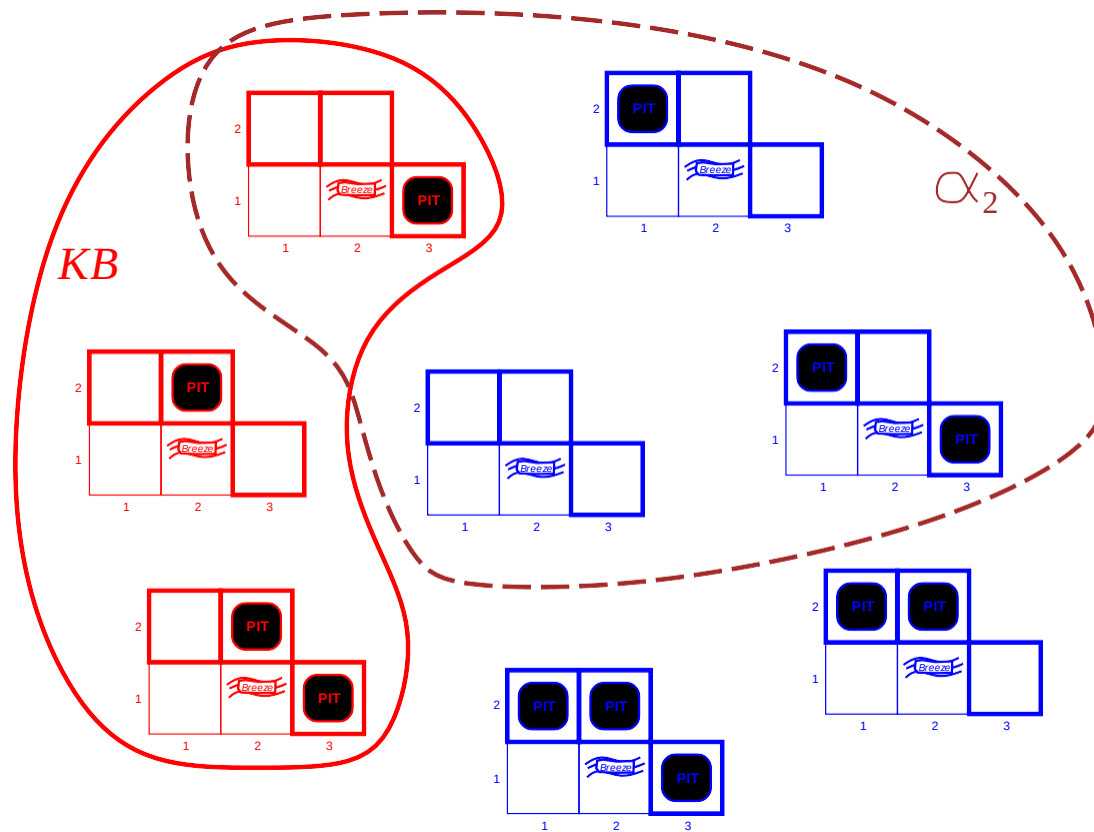
a_1 = "[1,2] is safe", $KB \models a_1$, proved by model checking

Wumpus models



KB = wumpus-world rules + observations

Wumpus models



KB = wumpus-world rules + observations

α_2 = “[2,2] is safe”, KB does not entail α_2

Model Checking



*enumerating models
and showing that the sentence must hold in all
models.*

Inference

the process of deriving conclusions from premises using valid reasoning.



$KB \vdash_i \alpha$

sentence α can be derived from KB by procedure i

Soundness: i is sound if

whenever $KB \vdash_i \alpha$, it is also true that $KB \models \alpha$

Completeness: i is complete if

whenever $KB \models \alpha$, it is also true that $KB \vdash_i \alpha$

Propositional logic: Syntax

Sentences are :

- Atomic = **proposition symbols** $P, Q \dots$
- Complex = constructed from simpler sentences using **logical connectives** ...

if S is a sentence, $\neg S$ is a sentence (**negation**)

If S_1 and S_2 are sentences, $S_1 \wedge S_2$ is a sentence (**conjunction**)

If S_1 and S_2 are sentences, $S_1 \vee S_2$ is a sentence (**disjunction**)

If S_1 and S_2 are sentences, $S_1 \Rightarrow S_2$ is a sentence (**implication**)

If S_1 and S_2 are sentences, $S_1 \Leftrightarrow S_2$ is a sentence (**biconditional**)

Propositional logic: Semantics

Each model specifies true/false for each proposition symbol

E.g. $P_{1,2}$ $P_{2,2}$ $P_{3,1}$
true true false

Rules for evaluating truth with respect to a model m :

$\neg S$ is true iff	S is false
$S_1 \wedge S_2$ is true iff	S_1 is true and S_2 is true
$S_1 \vee S_2$ is true iff	S_1 is true or S_2 is true
$S_1 \Rightarrow S_2$ is true iff	S_1 is false or S_2 is true
i.e., is false iff	S_1 is true and S_2 is false
$S_1 \Leftrightarrow S_2$ is true iff	$S_1 \Rightarrow S_2$ is true and $S_2 \Rightarrow S_1$ is true

Simple recursive process evaluates an arbitrary sentence, e.g.,

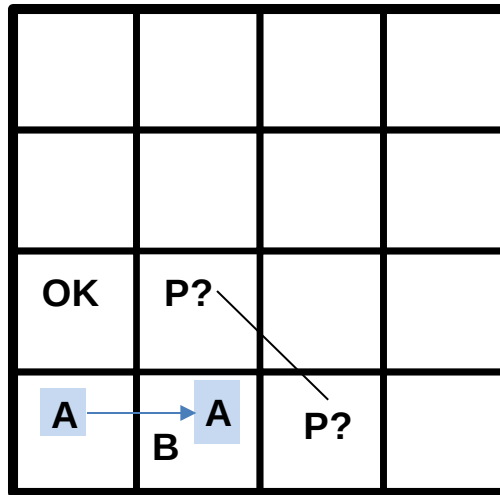
$\neg P_{1,2} \wedge (P_{2,2} \vee P_{3,1}) = \text{true} \wedge (\text{false} \vee \text{true}) = \text{true} \wedge \text{true} = \text{true}$

Truth tables for connectives

P	Q	$\neg P$	$P \wedge Q$	$P \vee Q$	$P \Rightarrow Q$	$P \Leftrightarrow Q$
<i>false</i>	<i>false</i>	<i>true</i>	<i>false</i>	<i>false</i>	<i>true</i>	<i>true</i>
<i>false</i>	<i>true</i>	<i>true</i>	<i>false</i>	<i>true</i>	<i>true</i>	<i>false</i>
<i>true</i>	<i>false</i>	<i>false</i>	<i>false</i>	<i>true</i>	<i>false</i>	<i>false</i>
<i>true</i>	<i>true</i>	<i>false</i>	<i>true</i>	<i>true</i>	<i>true</i>	<i>true</i>

Wumpus world sentences

Let $P_{i,j}$ be true if there is a pit in $[i,j]$.
 Let $B_{i,j}$ be true if there is a breeze in $[i,j]$.



R_1	$\neg P_{1,1}$
R_2	$B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$
R_3	$B_{2,1} \Leftrightarrow (P_{1,1} \vee P_{2,2} \vee P_{3,1})$
R_4	$\neg B_{1,1}$
R_5	$B_{2,1}$

“Pits cause breezes in adjacent squares”

Truth tables for inference

$B_{1,1}$	$B_{2,1}$	$P_{1,1}$	$P_{1,2}$	$P_{2,1}$	$P_{2,2}$	$P_{3,1}$	R_1	R_2	R_3	R_4	R_5	KB
false	false	false	false	false	false	false	true	true	true	true	false	false
false	false	false	false	false	false	true	true	true	false	true	false	false
.	.	.	.	.	.	.	.	.	.	.	.	.
false	true	false	false	false	false	false	true	true	false	true	true	false
false	true	false	false	false	false	true	true	true	true	true	true	<u>true</u>
false	true	false	false	false	true	false	true	true	true	true	true	<u>true</u>
false	true	false	false	false	true	true	true	true	true	true	true	<u>true</u>
false	true	false	false	true	false	false	true	false	false	true	true	false
.	.	.	.	.	.	.	.	.	.	.	.	.
true	true	true	true	true	true	true	false	true	true	false	true	false

A truth table constructed for the knowledge base given in the text. KB is true if R_1 through R_5 are true, which occurs in just 3 of the 128 rows (the ones underlined in the right-hand column). In all 3 rows, $P_{1,2}$ is false, so there is no pit in $[1,2]$. On the other hand, there might (or might not) be a pit in $[2,2]$.

Inference by enumeration

Depth-first enumeration of all models is sound and complete

```
function TT-Entails?(KB, a) returns true or false
  inputs: KB, the knowledge base, a sentence in propositional logic
         a, the query, a sentence in propositional logic
  symbols ← a list of the proposition symbols in KB and a
  return TT-Check-All(KB, a, symbols, [])
```

```
function TT-Check-All(KB, a, symbols, model) returns true or false
  if Empty?(symbols) then
    if PL-True?(KB, model) then return PL-True?(a, model)
    else return true
  else do
    P ← First(symbols); rest ← Rest(symbols)
    return (TT-Check-All(KB, a, rest, Extend(P, true, model)) and
           TT-Check-All(KB, a, rest, Extend(P, false, model)))
```

$O(2^n)$ for n symbols; problem is **co-NP-complete**

Propositional Theorem Proving



***applying rules of inference** directly to the sentences
in our knowledge base to construct a proof of the desired sentence **without consulting models.***

Logical equivalence

Two sentences are **logically equivalent** iff true in same models:

$a \equiv \beta$ if and only if $a \models \beta$ and $\beta \models a$

$$(a \wedge \beta) \equiv (\beta \wedge a) \quad \text{commutativity of } \wedge$$

$$(a \vee \beta) \equiv (\beta \vee a) \quad \text{commutativity of } \vee$$

$$((a \wedge \beta) \wedge \gamma) \equiv (a \wedge (\beta \wedge \gamma)) \quad \text{associativity of } \wedge$$

$$((a \vee \beta) \vee \gamma) \equiv (a \vee (\beta \vee \gamma)) \quad \text{associativity of } \vee$$

$$\neg(\neg a) \equiv a \quad \text{double-negation elimination}$$

$$(a \Rightarrow \beta) \equiv (\neg \beta \Rightarrow \neg a) \quad \text{contraposition}$$

$$(a \Rightarrow \beta) \equiv (\neg a \vee \beta) \quad \text{implication elimination}$$

$$(a \Leftrightarrow \beta) \equiv ((a \Rightarrow \beta) \wedge (\beta \Rightarrow a)) \quad \text{biconditional elimination}$$

$$\neg(a \wedge \beta) \equiv (\neg a \vee \neg \beta) \quad \text{De Morgan}$$

$$\neg(a \vee \beta) \equiv (\neg a \wedge \neg \beta) \quad \text{De Morgan}$$

$$(a \wedge (\beta \vee \gamma)) \equiv ((a \wedge \beta) \vee (a \wedge \gamma)) \quad \text{distributivity of } \wedge \text{ over } \vee$$

$$(a \vee (\beta \wedge \gamma)) \equiv ((a \vee \beta) \wedge (a \vee \gamma)) \quad \text{distributivity of } \vee \text{ over } \wedge$$

Some additional concepts...

Validity and satisfiability

A sentence is **valid** (a **TAUTOLOGY**) if it is true in **all** models

e.g., *True*, $A \vee \neg A$, $A \Rightarrow A$, $(A \wedge (A \Rightarrow B)) \Rightarrow B$

A sentence is **satisfiable** if it is true in **some** model

→ **SAT**
problem

e.g., $A \vee B, C$

A sentence is **unsatisfiable** (a **CONTRADICTION**) if it is true in **no** models

e.g., $A \wedge \neg A$

$KB \models a$ if and only if $(KB \wedge \neg a)$ is unsatisfiable

→ Proof by
contradiction

Deduction Theorem:

$KB \models a$ if and only if $(KB \Rightarrow a)$ is valid



$a \equiv b$ if and only if $a \models b$ and $b \models a$ if and only if $a \Rightarrow b$ and $b \Rightarrow a$ (i.e. $a \Leftrightarrow b$)

Proof methods

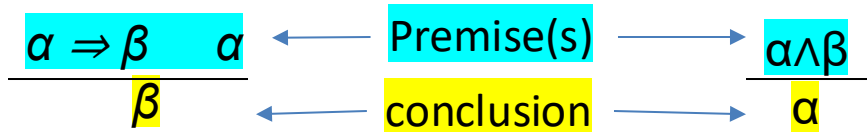
Application of inference rules

- Sound generation of new sentences
- **Proof** = a sequence of inference rule applications
- Typically require translation of sentences into a **normal form**

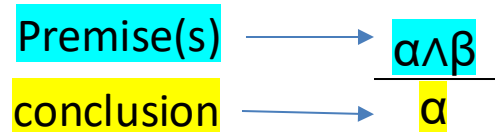
Model checking

- truth table enumeration (always exponential in n)
- improved backtracking, e.g., DPLL heuristic search in model space (sound but incomplete)

Modus Ponens



And-Elimination



Biconditional-elim

$$\frac{\alpha \Leftrightarrow \beta}{(\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)}$$

Contraposition

$$\frac{(\alpha \Rightarrow \beta)}{(\neg \beta \Rightarrow \neg \alpha)}$$

Unit Resolution

$$\frac{\alpha \vee \beta \quad \neg \alpha}{\beta}$$

De Morgan

$$\frac{\neg(\beta \vee \alpha)}{\neg \beta \wedge \neg \alpha}$$

A ok			

R_1	$\neg P_{1,1}$
R_2	$B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$
R_3	$B_{2,1} \Leftrightarrow (P_{1,1} \vee P_{2,2} \vee P_{3,1})$
R_4	$\neg B_{1,1}$

$$B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$$

Biconditional-elim

$$(B_{1,1} \Rightarrow (P_{1,2} \vee P_{2,1})) \wedge ((P_{1,2} \vee P_{2,1}) \Rightarrow B_{1,1})$$

And-Elimination

$$(P_{1,2} \vee P_{2,1}) \Rightarrow B_{1,1}$$

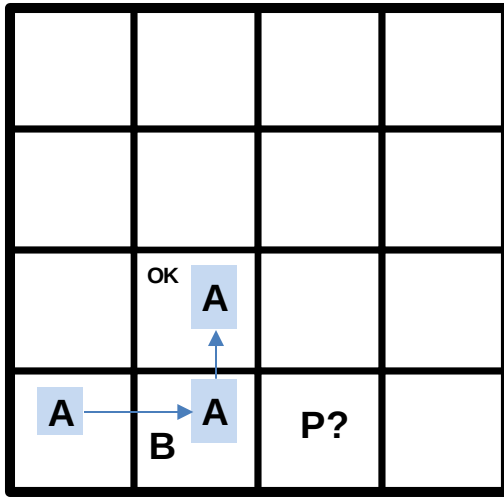
$$\neg B_{1,1} \Rightarrow \neg(P_{1,2} \vee P_{2,1}) \quad \text{Contraposition} \quad \neg B_{1,1}$$

Modus Ponens

$$\neg(P_{1,2} \vee P_{2,1})$$

De Morgan

$$\neg P_{1,2} \wedge \neg P_{2,1}$$



R_1	$\neg P_{1,1}$
R_2	$B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$
R_3	$B_{2,1} \Leftrightarrow (P_{1,1} \vee P_{2,2} \vee P_{3,1})$
R_4	$\neg B_{1,1}$
R_5	$\neg P_{1,2} \wedge \neg P_{2,1}$
R_6	$B_{2,1}$
R_7	$\neg P_{2,2}$

$B_{2,1} \Leftrightarrow (P_{1,1} \vee P_{2,2} \vee P_{3,1})$	Biconditional-elim
$(B_{2,1} \Rightarrow (P_{1,1} \vee P_{2,2} \vee P_{3,1})) \wedge ((P_{1,1} \vee P_{2,2} \vee P_{3,1}) \Rightarrow B_{2,1})$	And-Elimination
$(B_{2,1} \Rightarrow (P_{1,1} \vee P_{2,2} \vee P_{3,1}))$	Modus Ponens
$(P_{1,1} \vee P_{2,2} \vee P_{3,1})$	Unit Resolution
$(P_{1,1} \vee P_{3,1})$	Unit Resolution
$P_{3,1}$	

Conjunctive Normal Form (CNF)

conjunction of clauses

Clause = *disjunction of literals*

E.g., $(A \vee \neg B) \wedge (B \vee \neg C \vee \neg D)$

FACT:
Every sentence
of propositional logic is logically equivalent to a sentence in
CNF.

Conversion to CNF

$$B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$$

1. Eliminate $\Leftrightarrow$, replacing $a \Leftrightarrow b$ with $(a \Rightarrow b) \wedge (b \Rightarrow a)$.

$$(B_{1,1} \Rightarrow (P_{1,2} \vee P_{2,1})) \wedge ((P_{1,2} \vee P_{2,1}) \Rightarrow B_{1,1})$$

2. Eliminate $\Rightarrow$, replacing $a \Rightarrow b$ with $\neg a \vee b$.

$$(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg(P_{1,2} \vee P_{2,1}) \vee B_{1,1})$$

3. Move $\neg$ inwards using de Morgan's rules and double-negation:

$$(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge ((\neg P_{1,2} \wedge \neg P_{2,1}) \vee B_{1,1})$$

4. Apply distributivity law ($\vee$ over $\wedge$) and flatten:

$$(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg P_{1,2} \vee B_{1,1}) \wedge (\neg P_{2,1} \vee B_{1,1})$$

Conversion to CNF $(p \Rightarrow q) \Leftrightarrow (p \Rightarrow r)$

$$((p \Rightarrow q) \Rightarrow (p \Rightarrow r)) \wedge ((p \Rightarrow r) \Rightarrow (p \Rightarrow q))$$

$\equiv$

$$(\neg(p \Rightarrow q) \vee (p \Rightarrow r)) \wedge (\neg(p \Rightarrow r) \vee (p \Rightarrow q))$$

$\equiv$

$$((p \wedge \neg q) \vee (\neg p \vee r)) \wedge ((p \wedge \neg r) \vee (\neg p \vee q))$$

$\equiv$

$$(((p \vee \neg p) \wedge (\neg q \vee \neg p)) \vee r) \wedge (((p \vee \neg p) \wedge (\neg r \vee \neg p)) \vee q)$$

$\equiv$

$$(\neg q \vee \neg p \vee r) \wedge (\neg r \vee \neg p \vee q)$$

Resolution Algorithm



Basic inference mechanism: Resolution Rule

$$\frac{J_1 \vee \dots \vee J_k \qquad m_1 \vee \dots \vee m_n}{J_1 \vee \dots \vee J_{i-1} \vee J_{i+1} \vee \dots \vee J_k \vee m_1 \vee \dots \vee m_{j-1} \vee m_{j+1} \vee \dots \vee m_n}$$

where J_i and m_j are complementary literals.

$$\frac{P_{1,3} \vee P_{2,2} \qquad \neg P_{2,2}}{P_{1,3}}$$

Resolution is **sound** and **complete** for propositional logic



A resolution-based theorem prover can, for any sentences α and β in propositional logic, decide whether $\alpha \models \beta$.

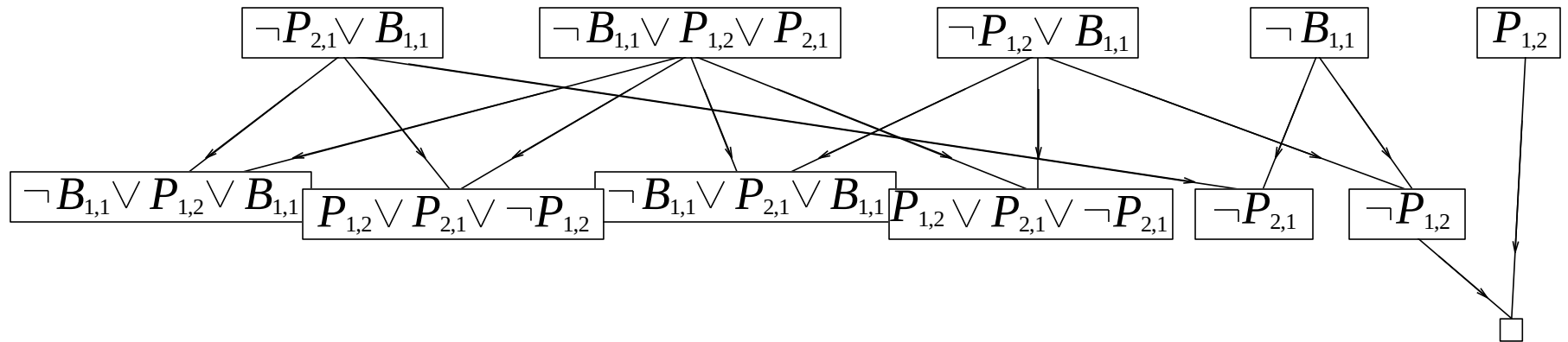
Resolution Algorithm

Proof by contradiction, i.e., show $KB \wedge \neg a$ unsatisfiable

```
function PL-Resolution( $KB, a$ ) returns true or false
  inputs:  $KB$ , the knowledge base, a sentence in propositional logic
          $a$ , the query, a sentence in propositional logic
   $clauses \leftarrow$  the set of clauses in the CNF representation of  $KB \wedge \neg a$ 
   $new \leftarrow \{ \}$ 
  loop do
    for each  $C_i, C_j$  in  $clauses$  do
       $resolvents \leftarrow$  PL-Resolve( $C_i, C_j$ )
      if  $resolvents$  contains the empty clause then return true
       $new \leftarrow new \cup resolvents$ 
    if  $new \subseteq clauses$  then return false
   $clauses \leftarrow clauses \cup new$ 
```

Resolution example

$$KB = (B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})) \wedge \neg B_{1,1} \quad a = \neg P_{1,2}$$



Ground resolution theorem

If a group of clauses is unsatisfiable, the empty clause is included in the resolution closure of those clauses.

(Completeness)

Horn Form

conjunction of Horn clauses

Horn clause = *disjunction of literals of which at most one is positive*.

$\neg A$
negative

A
positive

E.g., $C \wedge (B \Rightarrow A) \wedge ((C \wedge D) \Rightarrow B)$

$(a_1 \wedge \dots \wedge a_n \Rightarrow \beta) \wedge (c_1 \wedge \dots \wedge c_m \Rightarrow d)$

are in Horn form

$(A \vee \neg B) \wedge (B \vee C \vee \neg D)$

is not...

Chaining



Basic inference mechanism: Modus Ponens

$$\frac{a_1 \wedge \dots \wedge a_n \Rightarrow \beta \quad a_1 \wedge \dots \wedge a_n,}{\beta}$$

Applicable when KB is in
Horn Form... i.e. every
sentence is a Horn clause

Can be used with forward chaining or backward
chaining

in linear time

Forward chaining

Idea: If all the premises of an implication are known, then its conclusion is added to the set of known facts.

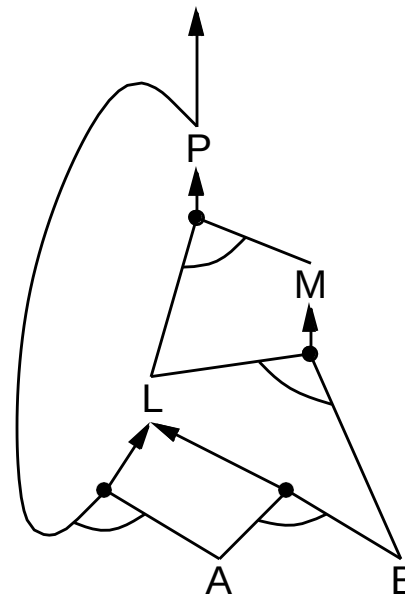
This process continues until the query q is added or until no further inferences can be made.

$$a_1 \wedge \dots \wedge a_n \Rightarrow \beta$$

Counter = number of symbols on left hand-side

If Counter = 0 Infer symbol on right-hand side, B

In **AND–OR graphs**, multiple edges joined by an arc indicate a **conjunction** while multiple edges without an arc indicate a **disjunction** -(see Chapter 4)

$$\begin{aligned}
 P &\Rightarrow Q \\
 L \wedge M &\Rightarrow P \\
 B \wedge L &\Rightarrow M \\
 A \wedge P &\Rightarrow L \\
 A \wedge B &\Rightarrow L \\
 A \\
 B
 \end{aligned}$$


Forward Chaining algorithm

- Maintain a current list of symbols from KB
- Initialize a counter for each clause in KB = number of symbols on the left-hand side
- Repeat:
 - Get the next symbol P from the queue
 - If $P = q$
 - We are done, $KB \models q$
 - Else
 - Decrease the counter of each clause in which P appear in the left-hand side
 - If the counter is 0: Add the right-hand side of the clause to the list

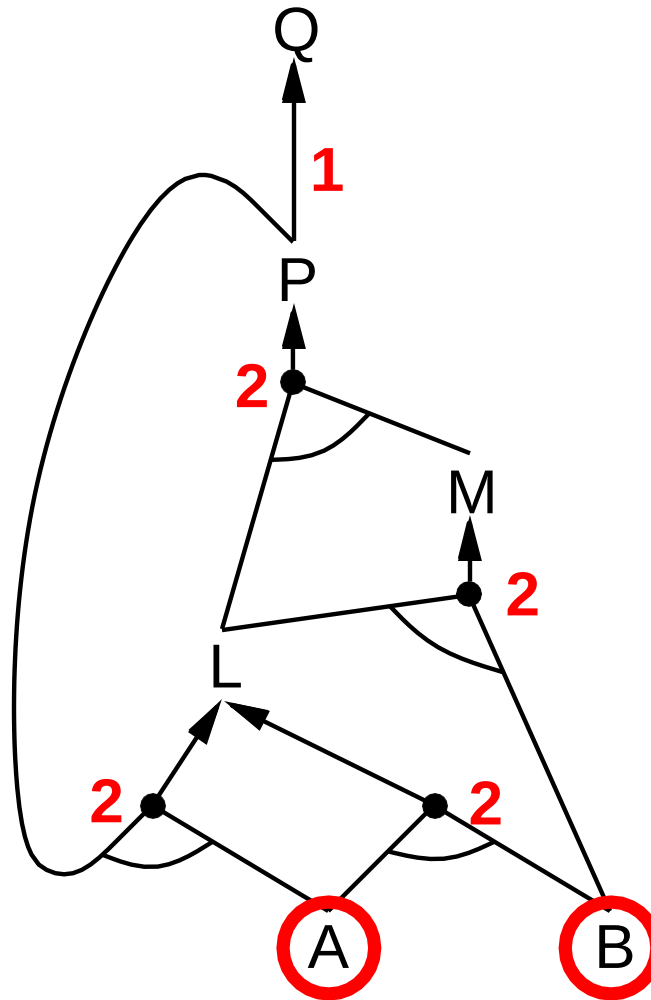
Forward chaining algorithm

```
function PL-FC-Entails?(KB, q) returns true or false
  inputs: KB, the knowledge base, a set of propositional Horn clauses
         q, the query, a proposition symbol
  local variables: count, a table, indexed by clause, initially the number of premises
                  inferred, a table, indexed by symbol, each entry initially false
                  agenda, a list of symbols, initially the symbols known in KB

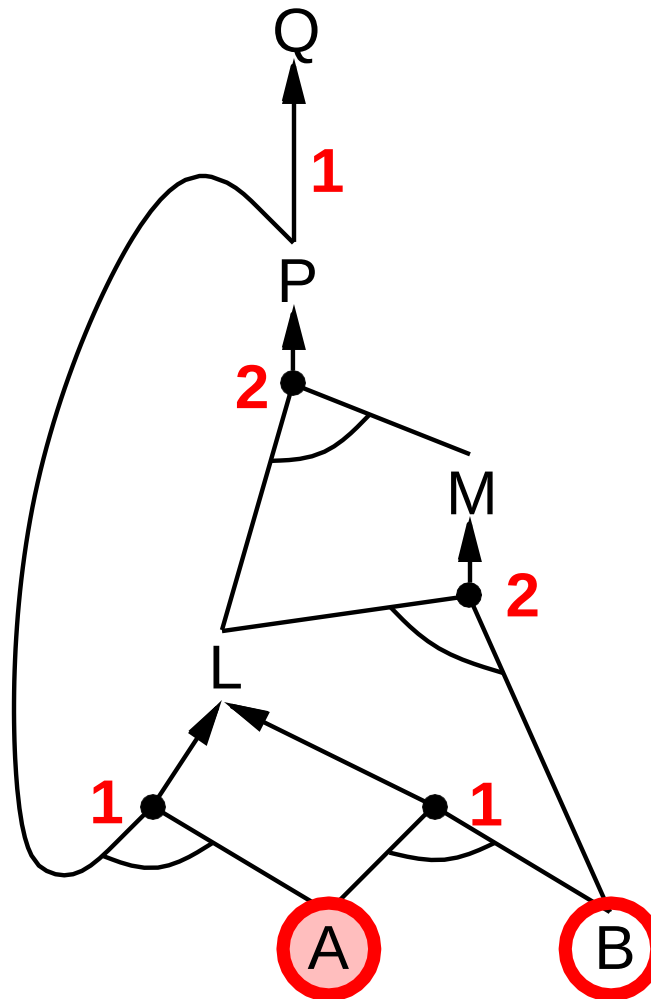
  while agenda is not empty do
    p ← Pop(agenda)
    unless inferred[p] do
      inferred[p] ← true
      for each Horn clause c in whose premise p appears do
        decrement count[c]
        if count[c] = 0 then do
          if Head[c] = q then return true
          Push(Head[c], agenda)

  return false
```

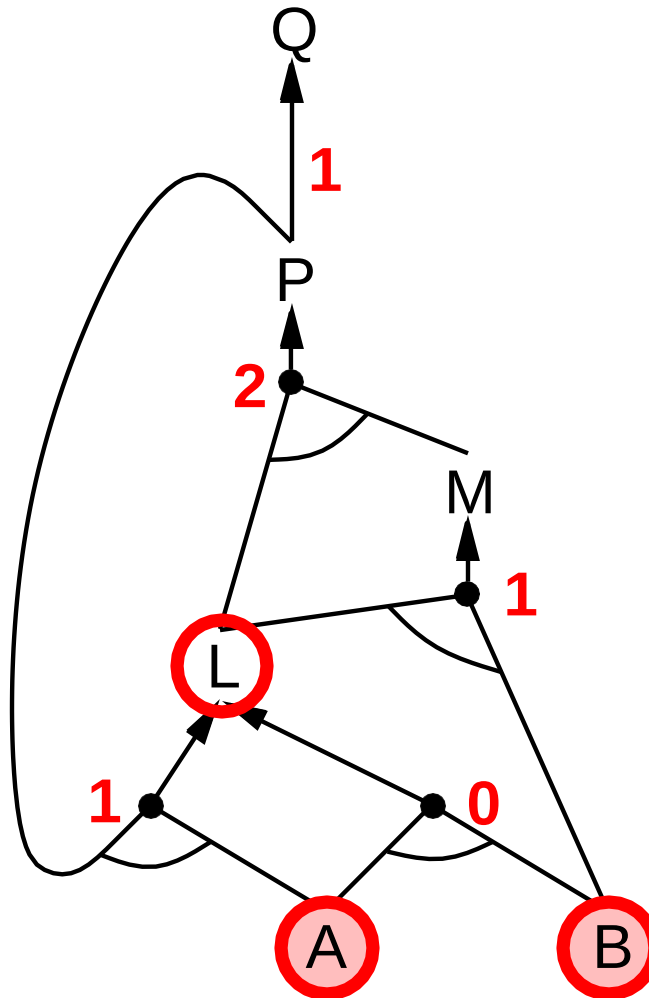
Forward chaining example



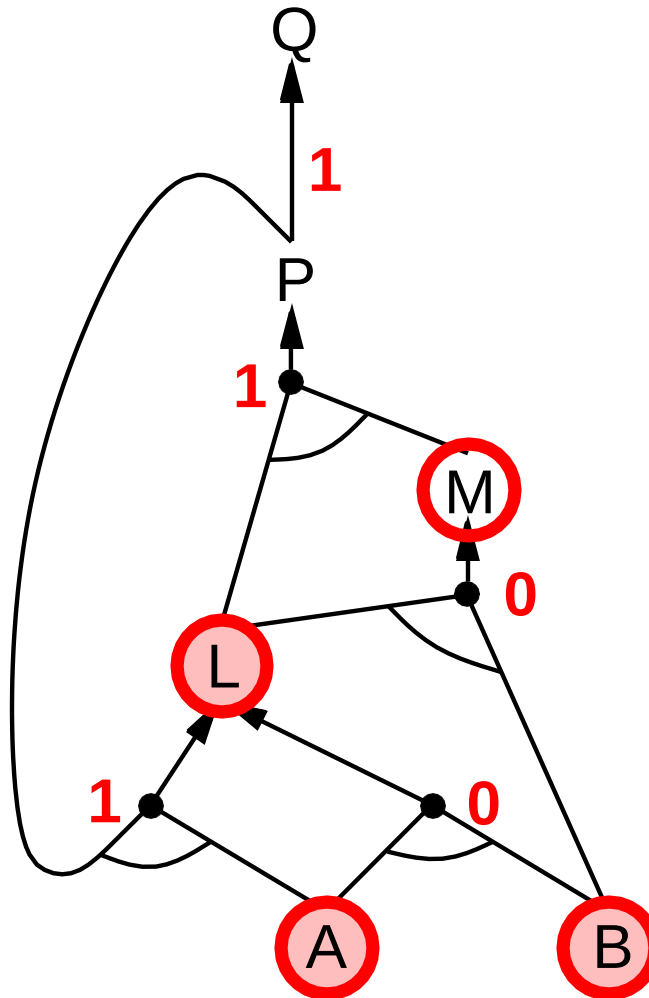
Forward chaining example



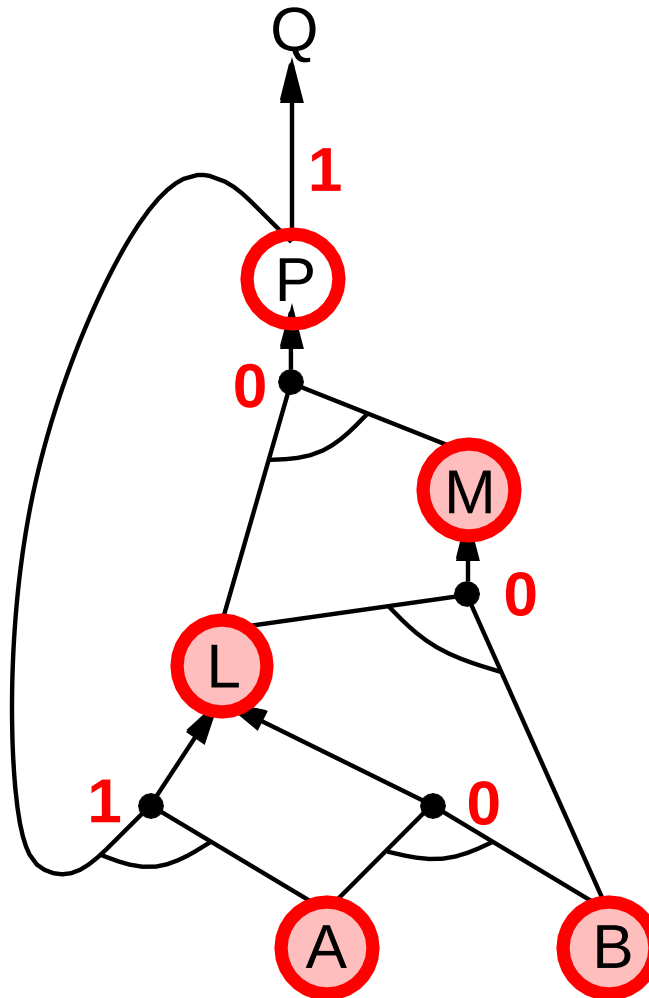
Forward chaining example



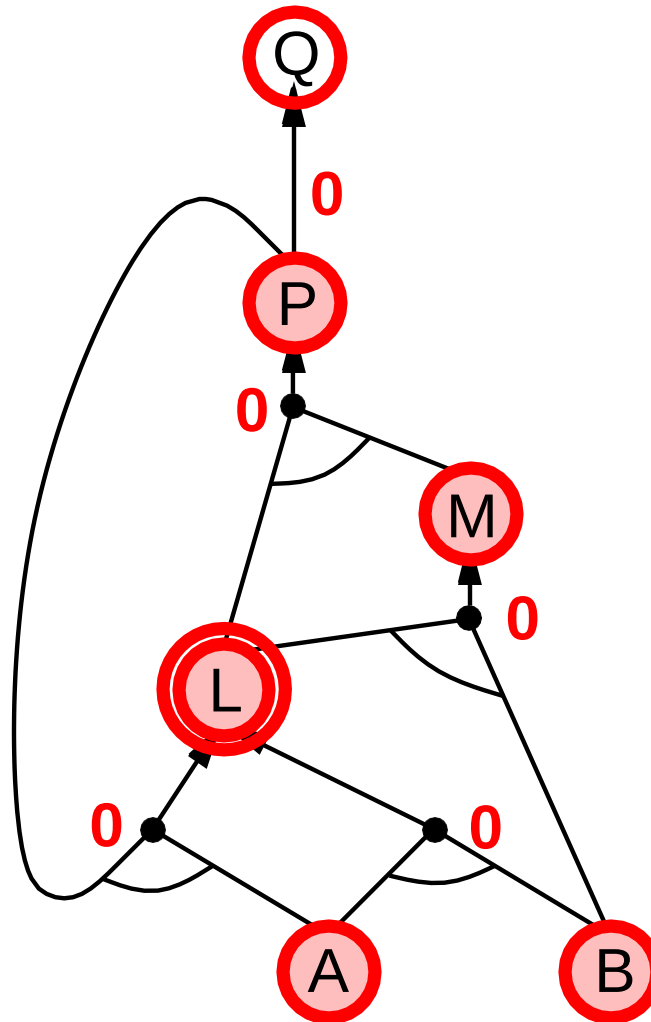
Forward chaining example



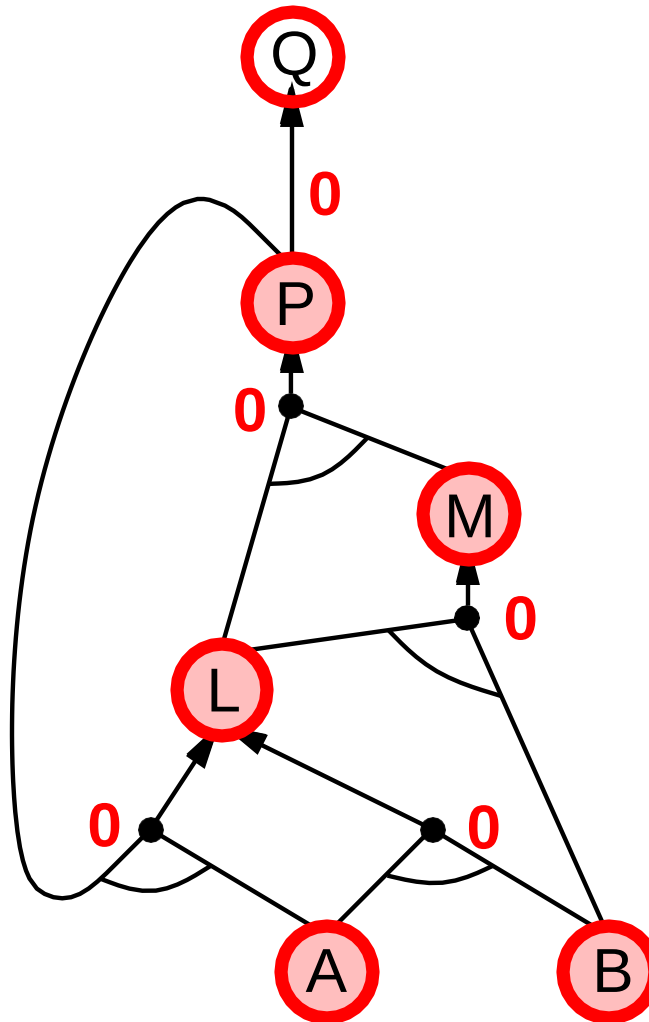
Forward chaining example



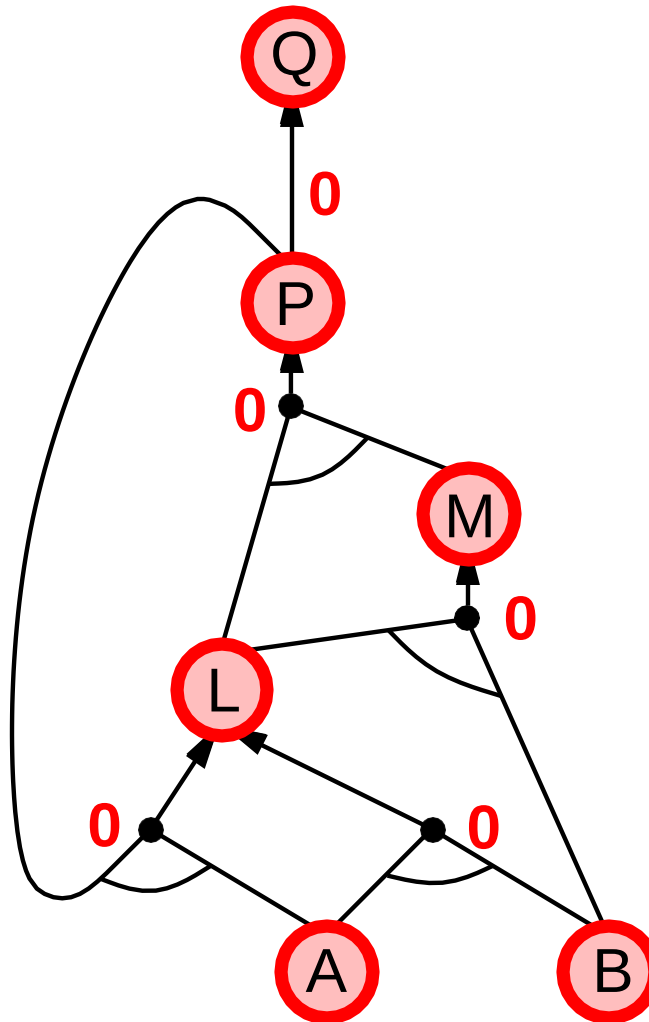
Forward chaining example



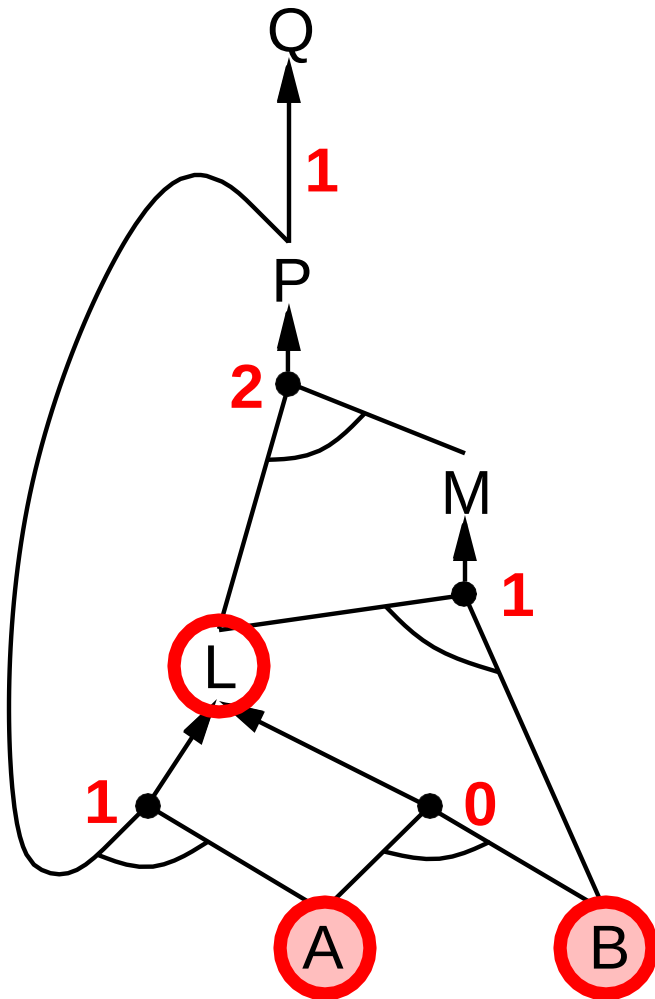
Forward chaining example



Forward chaining example



Soundness



$$\frac{A \wedge B \Rightarrow L \quad A \wedge B}{L}$$

Modus ponens

Modus Ponens is sound:

$$A \wedge B \Rightarrow L, A \wedge B \models L$$

Proof of completeness

FC derives every atomic sentence that is entailed by KB

1. FC reaches a **fixed point** where no new atomic sentences are derived
2. Consider the final state as a model m , assigning true/false to symbols

3. Every clause in the original KB is true in m

Proof: Suppose a clause $a_1 \wedge \dots \wedge a_k \Rightarrow b$ is false in m . Then $a_1 \wedge \dots \wedge a_k$ is true in m and b is false in m . Therefore the algorithm has not reached a fixed point!

4. Hence m is a model of KB

5. If $KB \models q$, q is true in **every** model of KB , including m

General idea: construct any model of KB by sound inference, check a

Backward chaining

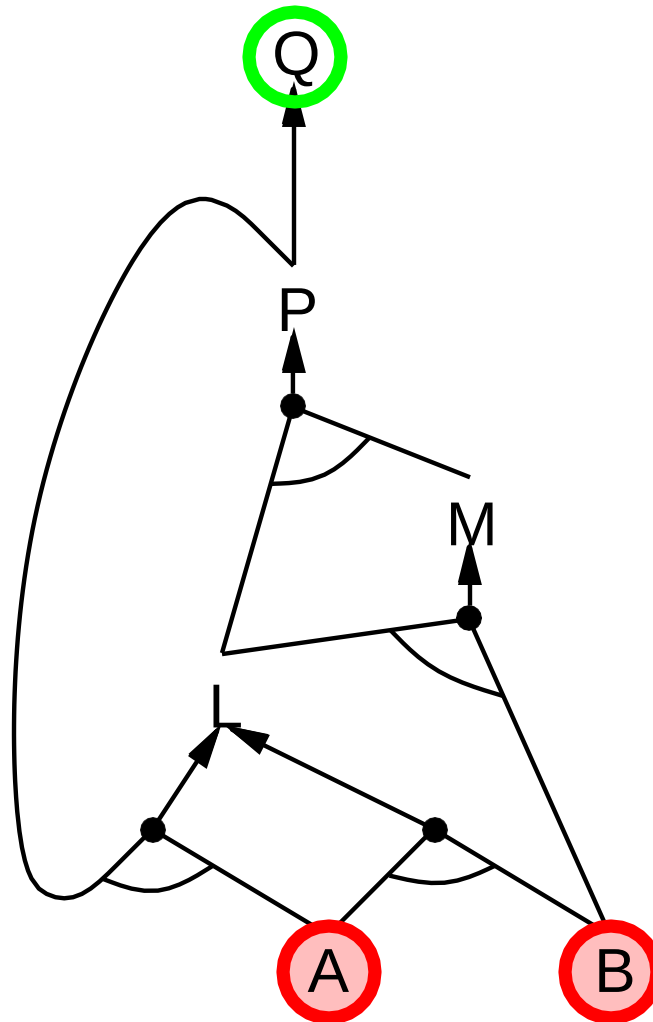
Idea :If the query q is known to be true, then no work is needed. Otherwise, the algorithm finds those implications in the knowledge base whose conclusion is q . If all the premises of one of those implications can be proved true (by backward chaining), then q is true.

Avoid loops: check if new subgoal is already on the goal stack

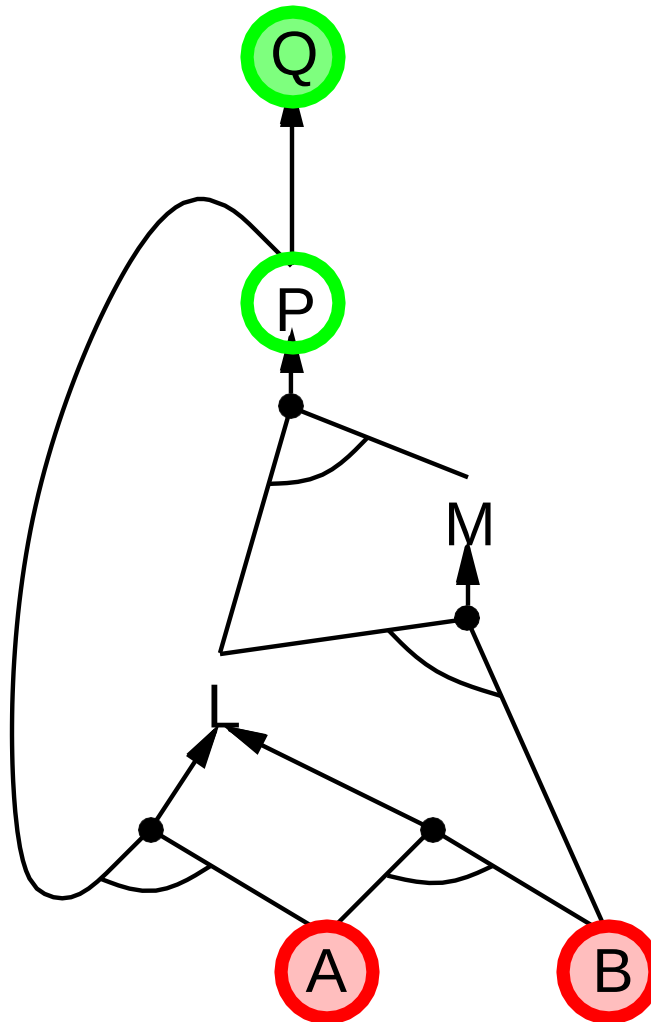
Avoid repeated work: check if new subgoal

- 1) has already been proved true, or
- 2) has already failed

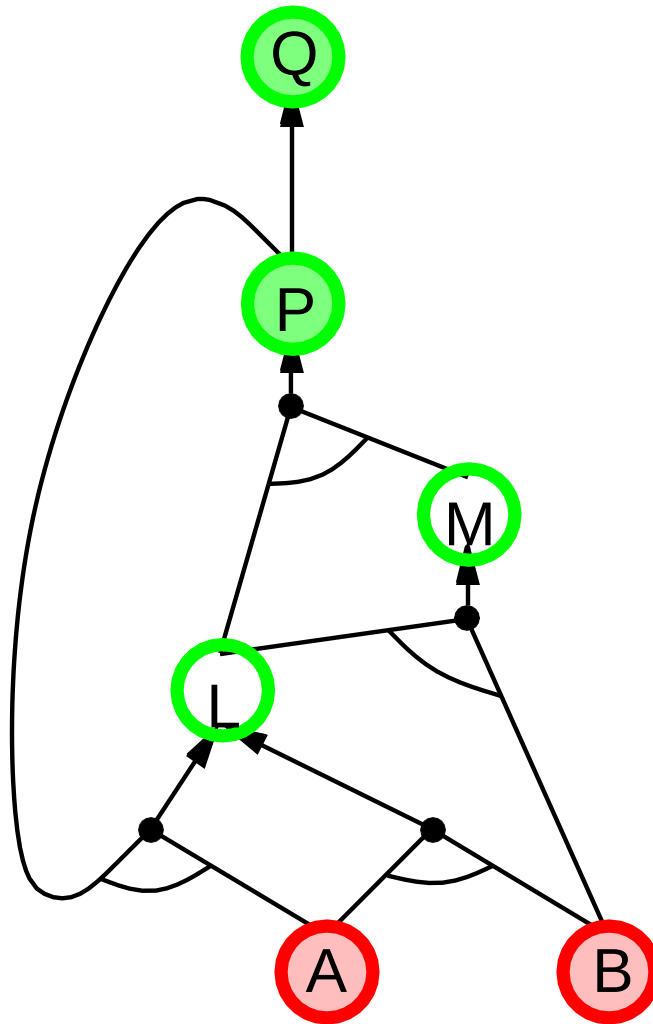
Backward chaining example



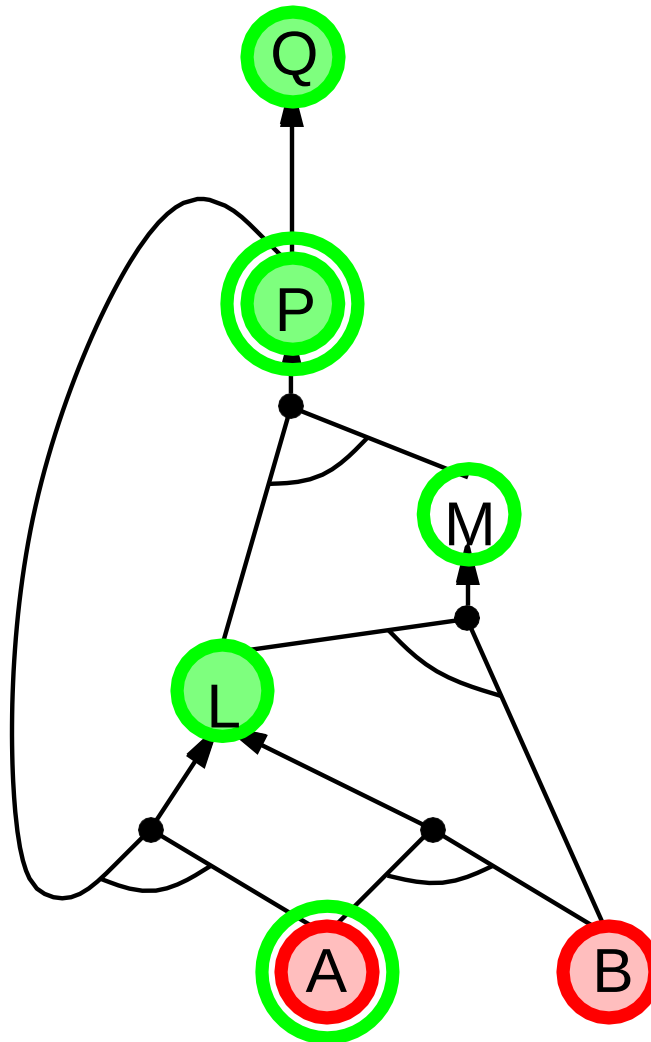
Backward chaining example



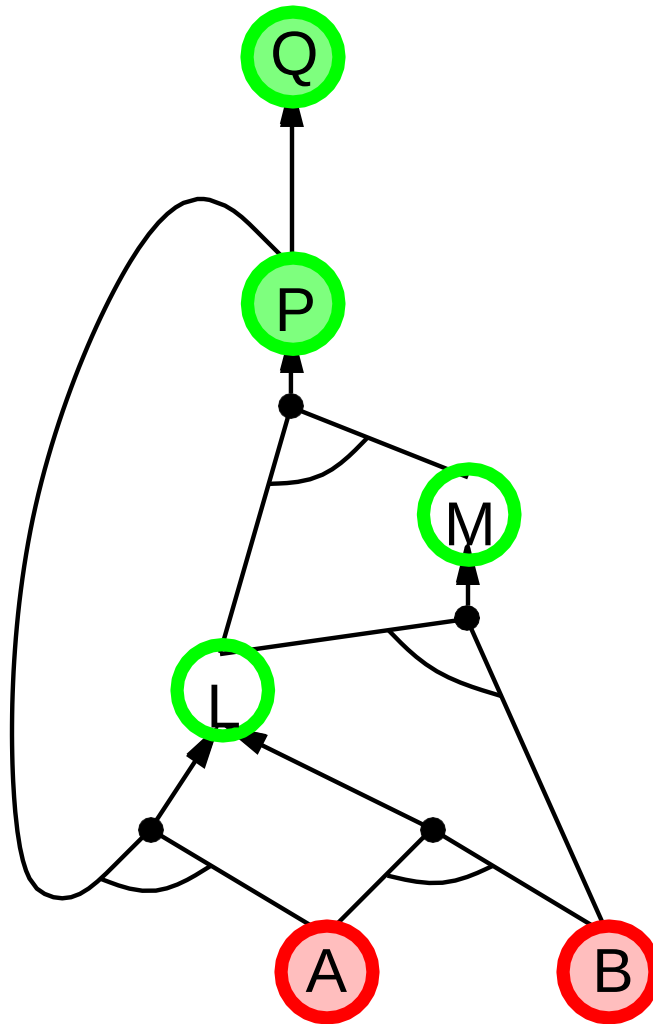
Backward chaining example



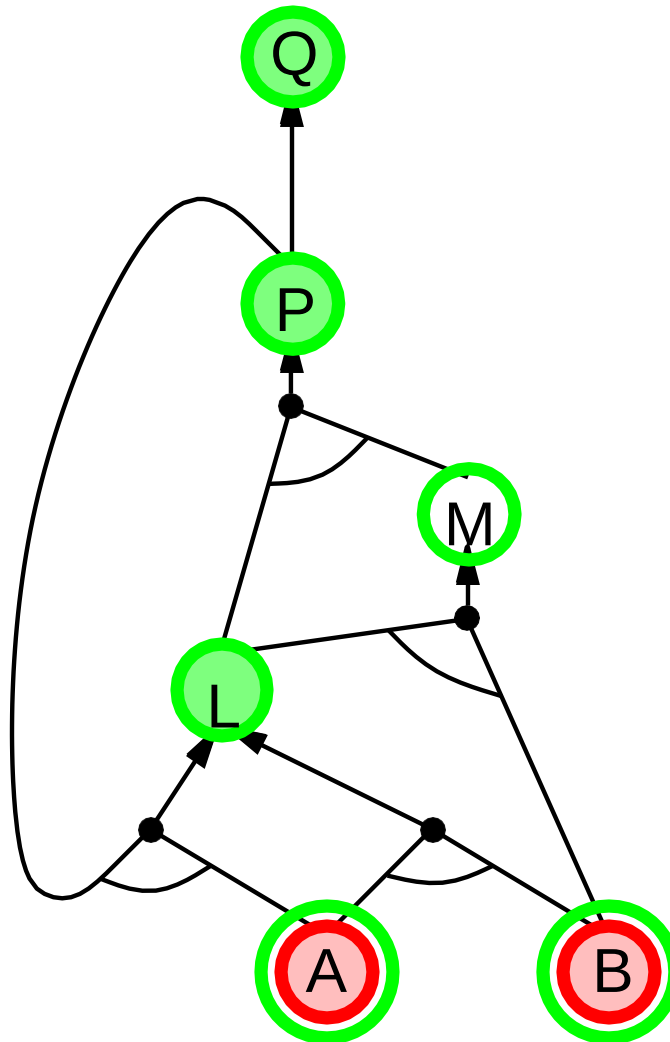
Backward chaining example



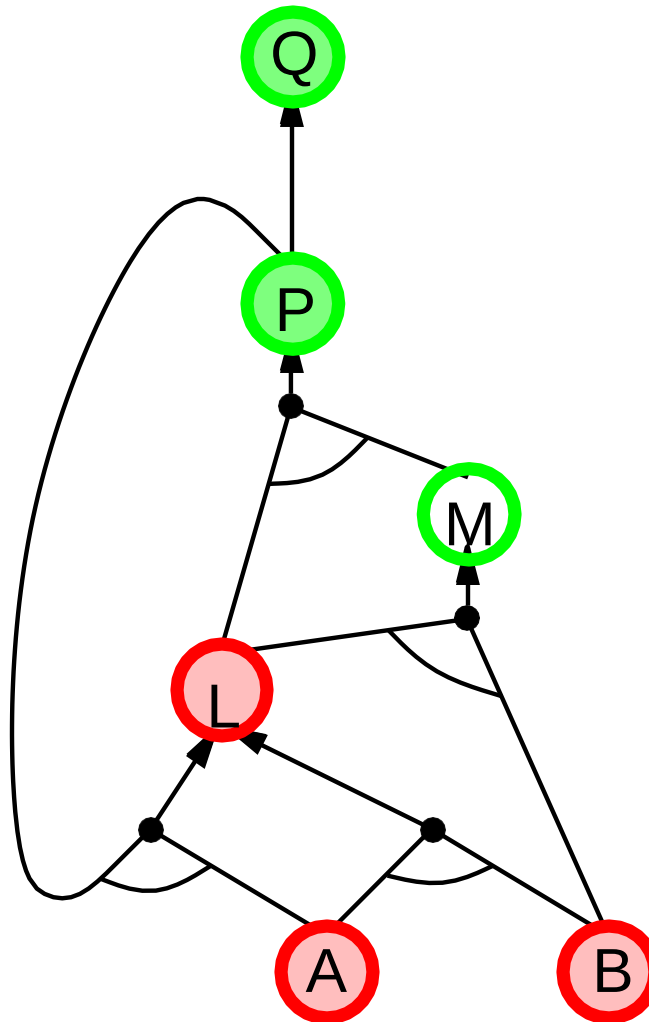
Backward chaining example



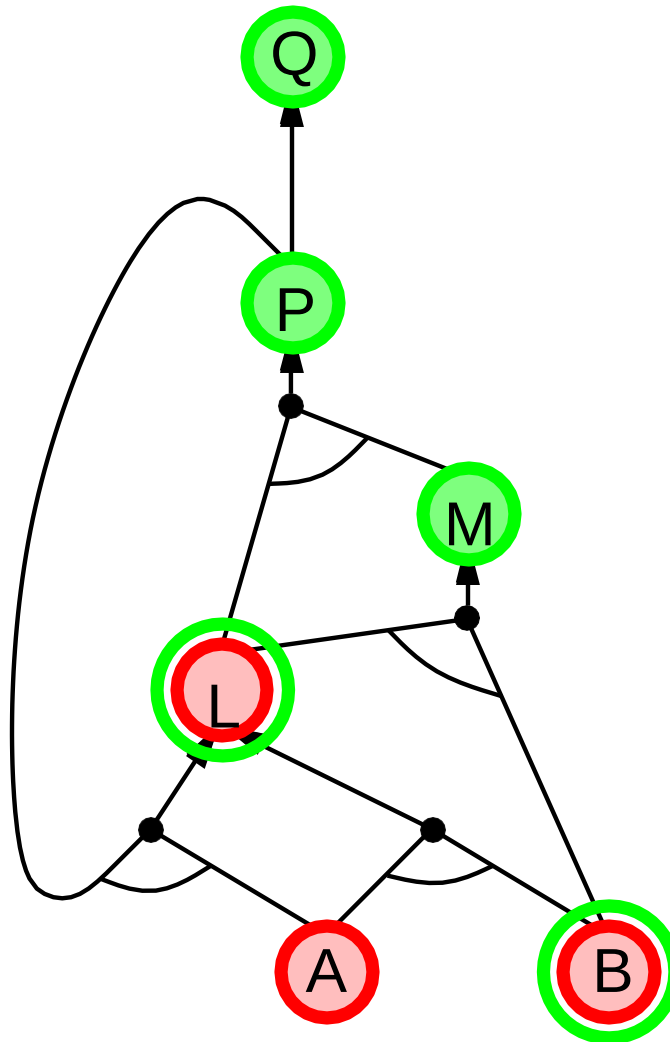
Backward chaining example



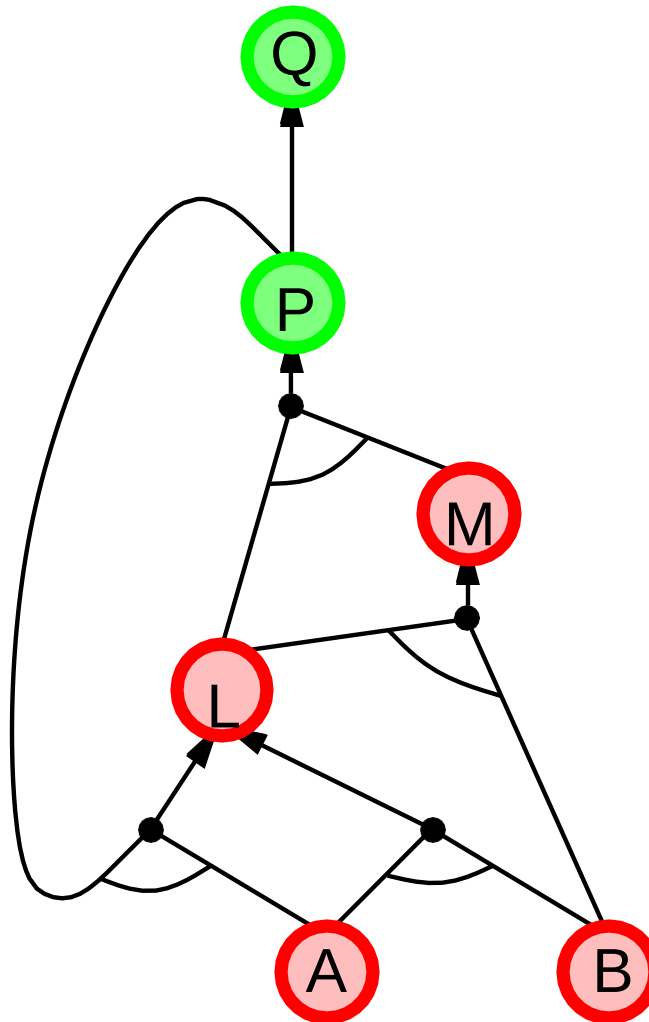
Backward chaining example



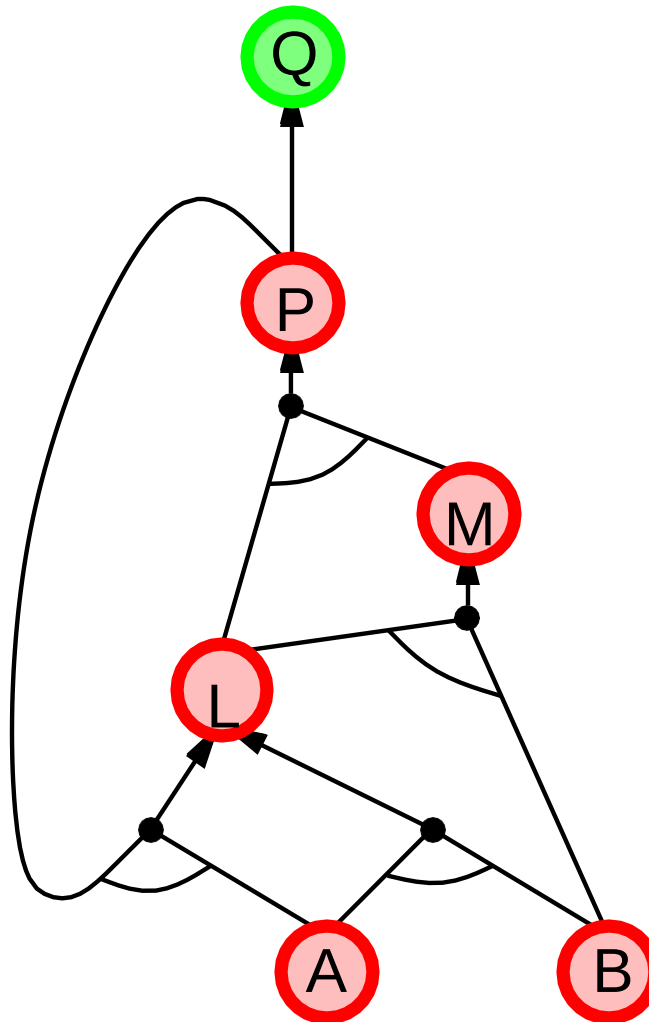
Backward chaining example



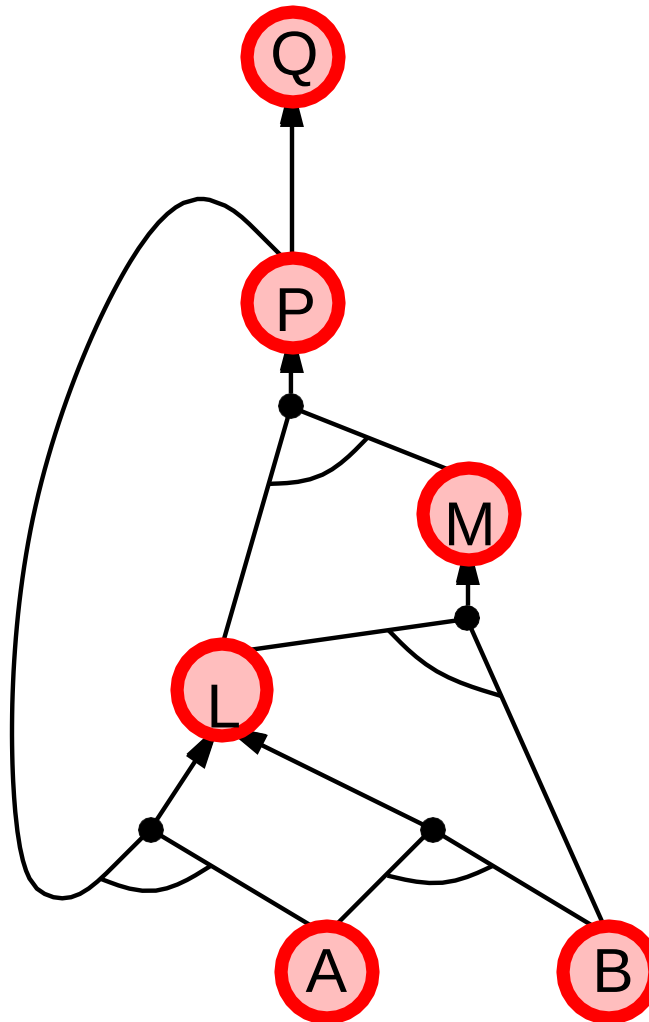
Backward chaining example



Backward chaining example



Backward chaining example



Forward vs. backward chaining

Data-driven

automatic, unconscious
processing

e.g. object recognition,
routine decisions

lots of work that is irrelevant to the
goal

Goal-driven

appropriate for problem-
solving

e.g. where are my keys?

Complexity of BC can be **much**
less than linear in size of KB

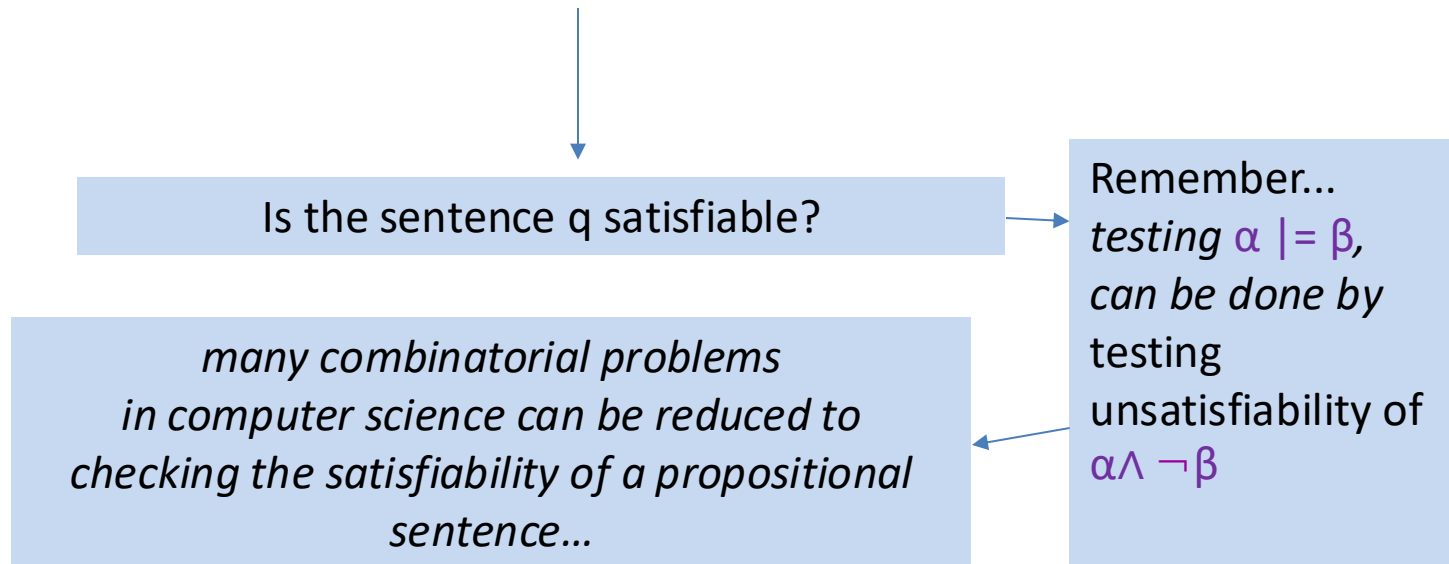
Sound, Complete and Linear

Effective Propositional Model Checking

An efficient algorithm for general propositional inference based on model checking

SAT Problem

A sentence is **satisfiable** if it is true in **some** model



$(\neg Q \vee \neg P \vee R) \wedge (\neg R \vee \neg P \vee Q)$ is SAT by all the models where p is false

$(P \vee Q) \wedge \neg P \wedge \neg Q$ is NOT SAT

Davis–Putnam algorithm (DPLL)



Inputs a sentence in CNF



a recursive, depth-first enumeration of possible models.
(similar to BACKTRACKING-SEARCH)



Returns true or false

Davis–Putnam algorithm (DPLL)

Improvements over TT-ENTAILS?:

- **Early termination:**
The algorithm detects whether the sentence is true or false, even with a partially completed model. Ex. $(A \vee B) \wedge (A \vee C)$
- **Pure symbol heuristic:**
 - symbol that always appears with the same **Pure symbol** “sign” in all clauses.Ex. In $(A \vee \neg B)$, $(A \vee \neg C)$, $(\neg B \vee C)$ the symbols A and B are pure, C is not.
 - ignore clauses that are already known.Ex. $(\neg B \vee \neg C)$, C if the first is true because B is false, then the first is ignored and C is considered pure.
- **Unit Clause heuristic:**
unit clause = with one literal OR clauses in which all literals but one are already assigned false by the model
Ex. if the model contains $B = \text{true}$, then $(\neg B \vee \neg C)$ simplifies to $\neg C$, which is a unit clause.

A simple DPLL algorithm

function **DPLL-SATISFIABLE?**(*s*) **returns** *true* or *false*

inputs: *s*, a sentence in propositional logic

clauses $\leftarrow$ the set of clauses in the CNF representation of *s*

symbols $\leftarrow$ a list of the proposition symbols in *s*

return **DPLL**(*clauses*, *symbols*, {})

function **DPLL**(*clauses*, *symbols*, *model*) **returns** *true* or *false*

if every clause in *clauses* is *true* in *model* then **return** *true*

if some clause in *clauses* is *false* in *model* then **return** *false*

P, *value* $\leftarrow$ **FIND-PURE-SYMBOL**(*symbols*, *clauses*, *model*)

if *P* is non-null then **return** **DPLL**(*clauses*, *symbols* – *P*, *model* $\cup$ {*P*=*value*})

P, *value* $\leftarrow$ **FIND-UNIT-CLAUSE**(*clauses*, *model*)

if *P* is non-null then **return** **DPLL**(*clauses*, *symbols* – *P*, *model* $\cup$ {*P*=*value*})

P $\leftarrow$ **FIRST**(*symbols*); *rest* $\leftarrow$ **REST**(*symbols*)

return **DPLL**(*clauses*, *rest*, *model* $\cup$ {*P*=*true*}) or

DPLL(*clauses*, *rest*, *model* $\cup$ {*P*=*false*})

Some DPLL tricks

- **Component analysis:**
the set of clauses may become separated into disjoint components
– gain speed
- **Variable and value ordering:**
choosing the variable that appears most frequently over all remaining clauses. (**degree heuristic**)
- **Intelligent backtracking:**
that backs up all the way to the relevant point of conflict.
- **Random restarts:**
Sometimes a run appears not to be making progress. In this case, we can start over
- **Clever indexing:**
fast indexing of such things as “the set of clauses in which variable X_i appears as a positive literal.”

Summary and reading exercise

Logical agents apply **inference** to a **knowledge base** to derive new information and make decisions

Basic concepts of logic:

- **syntax**: formal structure of **sentences**
- **semantics**: **truth** of sentences wrt **models**
- **entailment**: necessary truth of one sentence given another
- **inference**: deriving sentences from other sentences
- **soundness**: derivations produce only entailed sentences
- **completeness**: derivations can produce all entailed sentences

Forward, backward chaining are linear-time, complete for Horn clauses
Resolution is complete for propositional logic.

DPLL algorithm for SAT solving

Read from 7.1 to 7.6.1 of Artificial Intelligence - A modern approach